

Fortress: Multi-Level Secure and Efficient Distributed Learning

Hao Bai^{ID}, Graduate Student Member, IEEE, Kun He^{ID}, Member, IEEE, Yuqing Li^{ID}, Member, IEEE, Jing Chen^{ID}, Junting Gao^{ID}, Graduate Student Member, IEEE, and Ruiying Du^{ID}

Abstract—Distributed learning enables collaborative model training while preserving data locality, but faces critical security threats at three levels: gradient-level reconstruction, model-level inference, and training-level manipulation. Existing defenses target each level through secure aggregation, differential privacy, and malicious resilience respectively. However, combining them is challenging because secure aggregation demands encrypted computation, differential privacy requires prohibitively expensive nonlinear operations in encrypted domains, while malicious resilience necessitates costly verification protocols to ensure protocol correctness. In this paper, we present Fortress, a practical framework that addresses these conflicts by co-designing three mechanisms. Fortress implements secure aggregation via dual-server Boolean secret sharing, incorporates differential privacy through direct noise addition on secret shares using polynomial approximation to avoid expensive nonlinear computations, and ensures malicious resilience with lightweight transcript-based verification. Evaluation across multiple datasets demonstrates that Fortress achieves comprehensive multi-level security while reducing communication overhead by $6.2\times \sim 32.3\times$ and maintaining robustness under malicious attacks.

Index Terms—Distributed learning, privacy-preserving machine learning, secure aggregation.

I. INTRODUCTION

DISTRIBUTED learning enables multiple clients to collaboratively train machine learning models while keeping training data locally, addressing the growing need for privacy-preserving collaborative intelligence [1], [2]. This paradigm has become essential in numerous applications spanning health-care [3], finance [4], and edge computing [5], where data centralization is prohibited by privacy regulations [6].

However, distributed learning systems expose distinct security vulnerabilities across different operational phases that threaten the security of the collaborative training process. (1) *Gradient-level data reconstruction*: Shared local gradient updates leak information about local training data, enabling adversaries to perform gradient inversion attacks that reconstruct

individual samples [7], [8]. (2) *Model-level membership inference*: The aggregated global model becomes vulnerable to membership inference attacks. In such attacks, adversaries can determine whether specific data samples were used in training by analyzing model outputs [9], [10]. (3) *Training-level malicious manipulation*: Malicious clients can deliberately deviate from protocol specifications or inject invalid data to disrupt the distributed learning process, causing training failures or compromising the correctness of results [11], [12]. These multi-level vulnerabilities necessitate comprehensive protection mechanisms that address gradient privacy, model security, and training robustness simultaneously.

Existing secure distributed learning approaches target these threats through individual mechanisms, but each suffers from significant limitations when deployed. Secure aggregation approaches protect against gradient-level data reconstruction using secure multiparty computation or homomorphic encryption [13], [14]. However, they face scalability barriers due to expensive operations on high-precision encrypted values and computational complexity that grows substantially with model dimensions. Differential privacy (DP) techniques prevent model-level membership inference by adding calibrated noise [15], [16]. However, the noise scale required for strong privacy guarantees often severely degrades model convergence and utility. Noise accumulation from multiple sources further exacerbates these issues in distributed settings. Malicious resilience mechanisms detect training-level manipulation that deviates from or disrupts protocol execution [17], [18]. However, they require expensive cryptographic proofs and verification mechanisms, making them computationally inefficient.

Given that distributed learning systems face simultaneous threats across multiple security levels, it becomes natural and necessary to combine these approaches. However, when directly layering these methods on top of each other, each mechanism operates independently with its own requirements. This leads to multiplicative communication and computational overhead. *Challenge 1: Communication efficiency requirements for mechanism combination conflict with cryptographic operations*. Effective combination with DP and malicious resilience requires secure aggregation to first reduce its communication overhead, since additional costs from these mechanisms will compound on the already expensive secure aggregation. However, secure aggregation requires all gradient to be processed in full-precision encrypted format with fixed-width representations [19], [20], [21]. This conflicts with gradient compression techniques that

Received 10 March 2026; revised 15 June 2026; accepted 18 July 2026. Date of publication 21 July 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62302343 and Grant 62172303 and in part by the Key R&D Program of Hubei Province under Grant 2024BAB018. (Corresponding authors: Kun He; Yuqing Li.)

The authors are with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: bh0036@whu.edu.cn; hekun@whu.edu.cn; li.yuqing@whu.edu.cn; chenjing@whu.edu.cn; gao_jt@whu.edu.cn; duraying@whu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3715844

rely on variable bit-width encoding and adaptive quantization. *Challenge 2: Excessive computational complexity for DP within secure aggregation.* DP requires nonlinear operations: gradient norm computation, reciprocal calculation, and conditional clipping [22]. Each is expensive in the encrypted domain. Existing secure computation frameworks [23], [24] support basic arithmetic but not the sophisticated functions that DP noise calibration demands. Multiple rounds of interaction are therefore unavoidable under naïve composition, making the combined protocol impractical. *Challenge 3: Independent verification requirements across multiple mechanisms create multiplicative overhead.* Because secure aggregation correctness verification, DP noise calibration, and malicious behavior detection are based on different mathematical structures and security assumptions, they cannot be performed in a unified manner, forcing the system to maintain separate verification protocols for each mechanism. The result is prohibitive computational overhead that scales poorly with both the number of participants and model complexity. Without a unified design approach that addresses these conflicts, each security mechanism operates independently with separate overhead, making practical deployment challenging for large-scale systems.

In this paper, we present **Fortress**, a secure distributed learning system that addresses all three security levels through a unified dual-server Boolean secret sharing architecture. Fortress enables coordinated implementation of multiple security mechanisms. The key insight is to build all three protection mechanisms on a shared cryptographic foundation. This avoids the multiplicative overhead that arises when independent security systems are layered on top of one another. Fortress implements secure aggregation via dual-server Boolean secret sharing, incorporates DP through direct noise addition on secret shares using polynomial approximation to avoid expensive nonlinear computations, and ensures malicious resilience with lightweight transcript-based verification. The coordinated design stems from using Boolean secret sharing as the common substrate. Compressed gradients are decomposed into Boolean shares for cryptographic protection. DP noise is generated and applied directly on these secret shares through server-side computation. Malicious behavior is detected by verifying the consistency of sharing operations through transcript hashing.

Fortress makes three key technical contributions that address design challenges in coordinating multiple security mechanisms: **(1) Adaptive gradient compression with Boolean secret sharing.** A conflict exists between gradient compression and cryptographic security: compression requires variable bit-widths and adaptive quantization, while secure computation demands fixed-width representations and uniform algebraic structures. We address this conflict by developing client-level adaptive compression followed by uniform-width Boolean decomposition. First, clients perform component-wise adaptive quantization where the client-level adaptive compression parameter induces a uniform per-client bit-width, typically 4–6 bits in practice, while stochastic quantization determines the compressed integer values. Next, each compressed value is decomposed into its binary representation with adaptive bit-widths. Finally, each bit is secret-shared using XOR operations between

TABLE I
COMPARISON OF SECURE DISTRIBUTED LEARNING SCHEMES WITH FOCUS ON SECURITY AND EFFICIENCY

Scheme	Grad. Priv.	Model Priv.	Train. Rob.	Comm. Eff.	Comp. Eff.	Trust Model
FedAvg [†] [25]	×	×	×	✓	✓	○
SecAgg [19]	✓	×	×	✓	✓	○
RoFL [26]	✓	×	✓	×	×	○
PrivateFL [15]	✓	✓	×	×	×	○
Prio+ [23]	✓	×	✓	✓	✓	○●
ELSA [24]	✓	×	✓	✓	✓	○●
Fortress	✓	✓	✓	✓✓	✓✓	○●

Grad. Priv.: Protection against gradient inversion attacks on individual updates.

Model Priv.: Protection against membership inference attacks on the model.

Train. Rob.: Defenses against malicious participants disrupting training process.

Comm. Eff.: Communication efficiency during distributed training.

Comp. Eff.: Computational efficiency for privacy-preserving operations.

[†] denotes a performance reference only and provides no security guarantees.

○ = Single-server; ○● = Two non-colluding servers; ✓ = Supported;

✓✓ = Highly efficient (at least 5× lower communication overhead than all baselines, and the lowest per-round execution time).

two servers, where one share goes to each server. This design reduces communication from 32 bits to 4–6 bits per element. The resulting Boolean shares serve as the foundation for both DP operations and malicious detection, eliminating separate encryption layers.

(2) *Server-side differential privacy through secure computation.* Implementing DP in encrypted domains creates computational barriers. Complex nonlinear operations like reciprocal square root for gradient clipping become expensive when executed through multi-round protocols. We overcome it using polynomial approximation techniques that operate directly on Boolean secret shares from step (1). Specifically, servers first convert Boolean shares to arithmetic shares using pre-computed correlations. Then they jointly evaluate polynomial approximations of reciprocal square root functions needed for gradient clipping. Finally, servers generate Gaussian noise within the secure computation and add it to clipped gradients. It transforms expensive interactive protocols into efficient local computations. The mechanism operates entirely within the existing Boolean sharing framework, requiring no additional cryptographic infrastructure.

(3) *Lightweight malicious security using transcript verification.* Achieving malicious security typically requires expensive zero-knowledge proofs or commitment schemes that multiply the overhead of Boolean sharing operations. Fortress exploits the deterministic nature of Boolean operations used in steps (1) and (2). Clients first generate their Boolean shares and correlations for the compression and DP operations. Then they compute collision-resistant hashes of their complete computation traces, including input gradients, random seeds, and all intermediate values. Servers receive these transcript digests before performing aggregation. During the aggregation process, servers calculate hashes of intermediate results from their actual computation. Before revealing outputs, servers verify that their calculated hashes match the transcript digests provided by the client. Any mismatch triggers protocol abortion. This strategy protects the shared Boolean computation infrastructure with only constant communication overhead.

Table I presents a comprehensive comparison of Fortress with existing secure distributed learning schemes. Unlike prior

schemes that instantiate each security mechanism independently, Fortress builds gradient privacy, differential privacy, and malicious resilience on a unified Boolean secret sharing substrate. This co-design eliminates multiplicative overhead and enables practical multi-level security at scale. Our main contributions are summarized as follows:

- We design Fortress, a secure distributed learning framework that achieves multi-level security through a unified dual-server Boolean secret sharing. Fortress avoids the multiplicative overhead of independent security mechanisms by building all protections on a cryptographic foundation.
- We develop coordinated security mechanisms where adaptive gradient compression provides Boolean representations for cryptographic protection. Server-side DP operates directly on secret shares through polynomial approximation. Transcript verification ensures the integrity of the shared Boolean computation infrastructure.
- We provide comprehensive experimental evaluation demonstrating significant efficiency improvements compared to existing approaches. The evaluation shows effective defense against multiple attack types and deployment feasibility for large-scale distributed learning scenarios.

II. PRELIMINARIES

A. Problem Setting

System Architecture. We adopt a dual-server architecture for distributed learning that balances security requirements with computational efficiency. The system comprises two types of entities: clients $\mathcal{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_n\}$ and two computation servers $\mathcal{S}_0$ and $\mathcal{S}_1$. Single-server architectures suffer from privacy vulnerabilities where server compromise immediately reveals all client data. Multi-server protocols with $n \geq 3$ servers provide stronger security through techniques like Shamir secret sharing, but incur $O(n^2)$ communication overhead for secure multiplication operations due to pairwise server interactions.

Distributed Learning Setting. We consider a distributed learning scenario where n clients $\mathcal{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_n\}$ each possess local dataset $\mathcal{D}_i$ and collaboratively train a global model with parameters $\theta \in \mathbb{R}^d$, where d denotes the parameter dimension. The training proceeds through iterative rounds. In round t , the servers coordinate to select a subset of clients $\mathcal{C}_t \subseteq \mathcal{C}$, and each selected client $\mathcal{C}_i \in \mathcal{C}_t$ computes a local gradient update $g_i^{(t)}$ based on the current global model $\theta^{(t)}$ and their local data $\mathcal{D}_i$. The servers then securely aggregate these local updates to compute $\bar{g}^{(t)} = \frac{1}{|\mathcal{C}_t|} \sum_{i \in \mathcal{C}_t} g_i^{(t)}$, and update the global model as $\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \bar{g}^{(t)}$ where η denotes the learning rate.

B. Security Guarantees and Threat Model

Adversarial Model and Corruption Patterns. We consider a threat model where corrupted clients are assumed to be malicious and at most one server is corrupt. We formalize the adversarial capabilities through a probabilistic polynomial time (PPT) adversary $\mathcal{A}$ that can statically corrupt system participants. The adversary's corruption capability is characterized by the tuple $(\mathcal{C}_{corrupt}, \mathcal{S}_{corrupt})$, where $\mathcal{C}_{corrupt} \subseteq \mathcal{C}$ represents

corrupted clients, and $\mathcal{S}_{corrupt} \in \{\emptyset, \{\mathcal{S}_0\}, \{\mathcal{S}_1\}\}$ represents the corrupted server set. Following established distributed trust assumptions [23], [24], we assume that both servers are not simultaneously compromised, as they typically operate in separate trust domains. This is practically achieved through organizational separation (e.g., enterprise and cloud provider), or independent cloud services. We consider two corruption scenarios with different security guarantees:

(1) *Malicious clients with semi-honest server:* Corrupted clients can arbitrarily deviate from the protocol, including sending incorrect messages or refusing participation. The corrupted server (if any) follows protocol specifications but attempts to infer information from observable data.

(2) *Malicious clients with malicious server:* Both corrupted clients and the corrupted server can arbitrarily deviate from the protocol. This represents the strongest adversarial setting where the malicious server may actively manipulate protocol execution to compromise privacy.

Security Properties: Our protocol provides the following formal guarantees:

- *Privacy:* For any PPT adversary $\mathcal{A}$ controlling $\mathcal{C}_{corrupt}$ and at most one server, the local updates $g_i^{(t)}$ of honest clients remain computationally private: $\text{View}_{\mathcal{A}}(\Pi) \approx_c \mathcal{S}(\mathcal{C}_{corrupt}, \bar{g}^{(t)}, \{\|g_i^{(t)}\|_2\}_{i \in \mathcal{C}_{corrupt}})$, where Π denotes our protocol, $\text{View}_{\mathcal{A}}(\Pi)$ represents the adversary's view during protocol execution, $\mathcal{S}$ is a simulator that uses only corrupted clients' inputs, the aggregate result $\bar{g}^{(t)}$, and public metadata such as gradient norm bounds, and $\approx_c$ denotes computational indistinguishability.
- *Robustness:* The protocol identifies and filters malicious updates from corrupted clients through verification predicates ϕ (such as ℓ_2 norm bounds). Updates satisfying $\phi(g_i^{(t)}) = 1$ are included in the aggregation while those with $\phi(g_i^{(t)}) = 0$ are rejected with high probability.
- *Correctness:* When at least one server is honest and sufficient verified client updates are available, the protocol outputs the correct result: $\Pr[\text{output} = \bar{g}^{(t)} \mid \text{sufficient honest clients}] \geq 1 - \text{negl}(\lambda)$, where λ is the security parameter and $\text{negl}(\lambda)$ denotes a negligible function.

Security Boundary Under Server Collusion: If both servers collude, the security guarantees degrade as follows. *Gradient-level privacy* is lost: a colluding pair can reconstruct any individual compressed gradient. *Model-level privacy* partially survives: Gaussian noise is incorporated into the aggregated output within the secret-shared domain and remains present in the final reconstruction. *Training-level privacy* is voided: colluding servers can pre-coordinate to forge consistent transcript hashes, bypass correlation verification, and choose not to enforce the ℓ_2 -norm bound on client shares. Under full collusion, the security level degrades to DP-only, equivalent to PrivateFL [15]. This degradation is inherent to the two-server model; it is not a unique limitation of Fortress.

C. Building Blocks

Secret Sharing: Secret sharing [27] enables the distribution of a secret value among multiple parties such that the secret

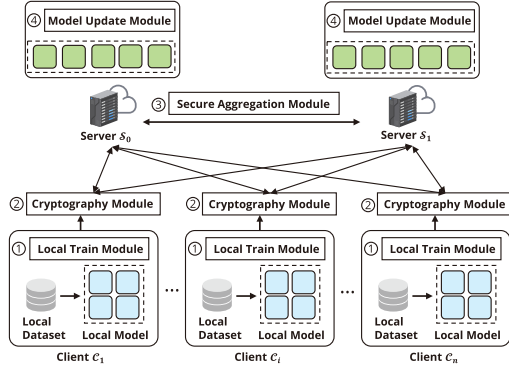


Fig. 1. Overview of the Fortress. Two servers S_0 and S_1 in separate trust domains coordinate secure aggregation. Clients C_i perform local training and use cryptography module to generate secret shares and correlations.

can only be reconstructed when a sufficient number of parties cooperate. In arithmetic secret sharing over a ring $\mathbb{Z}_q$, a secret value $x \in \mathbb{Z}_q$ is split into shares $\langle x \rangle^{(0)}, \langle x \rangle^{(1)}$ such that $x = \langle x \rangle^{(0)} + \langle x \rangle^{(1)} \pmod{q}$. Given arithmetic shares of values x and y , parties can locally compute shares of $x + y$ as $\langle x + y \rangle^{(i)} = \langle x \rangle^{(i)} + \langle y \rangle^{(i)} \pmod{q}$. Boolean secret sharing operates over $\mathbb{F}_2 = \{0, 1\}$ using XOR operations, where a secret bit b is shared as $b = \langle b \rangle^{(0)} \oplus \langle b \rangle^{(1)}$ and XOR operations can be computed locally as $\langle x \oplus y \rangle^{(i)} = \langle x \rangle^{(i)} \oplus \langle y \rangle^{(i)}$.

Oblivious Transfer: Oblivious Transfer (OT) [28], [29] enables a sender to transfer one of several pieces of information to a receiver, without the sender learning which piece was transferred. In 1-out-of-2 OT, the sender holds messages (m_0, m_1) and the receiver holds choice bit $c \in \{0, 1\}$. The receiver learns m_c while the sender learns nothing about c . OT extension techniques allow generating many OT instances from $O(\lambda)$ base OTs where λ is security parameter. In correlated OT (COT) [30], the sender's messages satisfy correlations like $m_1 = m_0 + \Delta$ for fixed offset Δ . Fortress uses Δ -COT correlations where correlation tuples (q_j, t_j) satisfy $t_j = q_j + c_j \cdot \Delta$ for choice bits c_j .

Differential Privacy: Differential privacy (DP) [22] provides privacy guarantees for randomized algorithms. An algorithm $\mathcal{M} : \mathcal{D}^n \rightarrow \mathcal{R}$ satisfies (ϵ, δ) -DP if for all adjacent datasets $\mathcal{D}, \mathcal{D}'$ and measurable subsets S : $\Pr[\mathcal{M}(\mathcal{D}) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(\mathcal{D}') \in S] + \delta$. Gaussian mechanism achieves DP by adding noise $\mathcal{M}(\mathcal{D}) = f(\mathcal{D}) + \mathcal{N}(0, \sigma^2 I_d)$ where $\sigma \geq \frac{\sqrt{2 \ln(1.25/\delta)} \cdot \Delta_2(f)}{\epsilon}$ and $\Delta_2(f)$ is ℓ_2 -sensitivity. Since gradients can have unbounded norms, gradient clipping $\text{clip}(g^{(t)}, C) = g^{(t)} \cdot \min(1, \frac{C}{\|g^{(t)}\|_2})$ is applied before noise addition to bound sensitivity to $2C$, where C is the clipping threshold.

III. DESIGN OF FORTRESS

Fig. 1 presents the Fortress framework architecture. Prior to training, Fortress executes a one-time setup where servers S_0 and S_1 negotiate security parameters and establish an authenticated channel. Clients then perform authenticated key exchange with both servers to secure communication (e.g., via TLS). Crucially, correlated randomness (e.g., PRG seeds) required for Boolean

secret sharing and Oblivious Transfer is pre-distributed during this phase, ensuring all entities are cryptographically prepared for the online operations. In training round t , servers S_0 and S_1 broadcast the global model $\theta^{(t)}$ to clients C_t . Clients perform local training on private datasets $\mathcal{D}_i$ to compute local gradient updates $g_i^{(t)}$, apply adaptive compression to reduce communication overhead while maintaining Boolean secret sharing compatibility, and generate cryptographic correlations (specifically Δ -COT correlations enabling efficient Boolean-to-arithmetic conversion and square correlations supporting secure ℓ_2 norm computation required for DP). These compressed gradients and correlations are distributed between servers, ensuring that neither server learns individual gradient information. Servers then execute the secure aggregation protocol: first verifying the correctness of client-provided correlations through batch verification techniques, then performing Boolean-to-arithmetic conversion using verified correlations, computing ℓ_2 norms for secure gradient clipping, and applying calibrated DP noise through secure computation. Upon completion of secure aggregation, servers jointly reconstruct the differentially private aggregate $\tilde{g}^{(t)}$ and update the global model as $\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \tilde{g}^{(t)}$.

A. Adaptive Gradient Compression With Boolean Secret Sharing

The motivation for gradient compression in secure distributed learning extends beyond bandwidth reduction. Distributed learning protocols require clients to transmit high-precision local gradient updates, resulting in communication costs that scale linearly with both model size and client count. In secure variants, gradient compression faces specific technical obstacles. First, compression alters the numerical precision and structure of transmitted data, which can interfere with verification mechanisms essential for secure computation. Second, secure multi-party computation relies on specific algebraic structures in finite field operations that must be preserved throughout computational phases.

Adaptive Gradient Compression: We design a compression mechanism that maintains compatibility with Boolean secret sharing while achieving significant communication reductions through adaptive compression that responds to gradient dynamics throughout the training process. Each client $i \in \mathcal{C}$ performs local training on private dataset $\mathcal{D}_i$ to obtain local gradient updates $g_i^{(t)}$, then applies our dynamic compression function with adaptive level $s_i^{(t)} \geq 1$ that automatically adjusts based on gradient stability across rounds. The adaptive mechanism employs the relationship $s_i^{(t)} = \alpha s_i^{(t-1)} + \beta \sqrt{\|g_i^{(t-1)}\|_2 - \|g_i^{(t)}\|_2}$ where hyperparameters $\alpha + \beta = 1$ control the balance between historical stability and current gradient variations. This is motivated by empirical observations that gradient norm changes correlate with the need for compression precision. During stable training phases with small gradient variations, higher compression maintains adequate reconstruction quality, while periods of rapid optimization require reduced compression to preserve information.

The square root function $\sqrt{\cdot}$ serves three critical theoretical purposes. First, it provides *variance stabilization* for the

Protocol 1: Adaptive Gradient Compression.

Input: Dataset $\mathcal{D}_i$ of client i , compression level $s_i^{(t)}$, hyperparameters α, β .

Output: Quantized integer gradients $\{\bar{g}_{i,j}^{(t)}\}_{j=1}^m$.

```

1:  $g_i^{(t)} \leftarrow \text{LocalTrain}(\mathcal{D}_i, \theta^{(t-1)})$ .
2:  $s_i^{(t)} \leftarrow \alpha s_i^{(t-1)} + \beta \sqrt{|\|g_{i,j}^{(t-1)}\|_2 - \|g_i^{(t)}\|_2|}$ .
3: for  $j \in [m]$  do
4:   Compute ratio:  $f_{i,j}^{(t)} \leftarrow \|g_{i,j}^{(t)}\|_2 / \|g_i^{(t)}\|_2 \times s_i^{(t)}$ .
5:   Extract integer part:  $l \leftarrow \lfloor f_{i,j}^{(t)} \rfloor$ .
6:   Sample probability:  $p \leftarrow f_{i,j}^{(t)} - l$ .
7:   Sample  $b \sim \text{Bernoulli}(p)$ . //  $b \in \{0, 1\}$ 
8:    $\xi_{i,j}^{(t)} \leftarrow l + b$ .
9:    $\bar{g}_{i,j}^{(t)} \leftarrow \text{sgn}(g_{i,j}^{(t)}) \times \xi_{i,j}^{(t)}$ . // Result is a signed integer
10: end for
11: return  $\{\bar{g}_{i,j}^{(t)}\}_{j=1}^m$ .
```

non-uniformly distributed gradient norm differences $|\|g_{i,j}^{(t-1)}\|_2 - \|g_i^{(t)}\|_2|$, which exhibit large variations especially during early training phases. The square root transformation reduces the impact of extreme values on compression levels, preventing over-compression during periods of high gradient volatility. Second, the *concavity* of the square root function ensures diminishing marginal effects of compression level changes, preventing oscillations caused by small gradient fluctuations while maintaining sensitivity to significant changes. Third, from *signal-to-noise ratio theory* in signal processing, the square root transformation maximizes the signal-to-noise ratio when gradient changes follow certain distributions, preserving important gradient information while reducing noise effects.

Compression Distribution and Two's Complement Representation: The compression operates by mapping each gradient component $g_{i,j}^{(t)}$ to a discrete integer set $\mathbb{Z} \cap [-s_i^{(t)}, s_i^{(t)}]$ using stochastic quantization. Mapping gradients directly to a large prime field $\mathbb{Z}_p$ introduces representation overhead for negative values. To avoid this, we employ a randomized quantization scheme that produces low-bitwidth signed integers. First, we compute the normalized magnitude ratio $f_{i,j}^{(t)} = \|g_{i,j}^{(t)}\|_2 / \|g_i^{(t)}\|_2 \times s_i^{(t)}$. Let $\lfloor f_{i,j}^{(t)} \rfloor$ denote the integer part. We sample a random variable $\xi_{i,j}^{(t)}$ via a Bernoulli trial such that $\xi_{i,j}^{(t)} = \lfloor f_{i,j}^{(t)} \rfloor + 1$ with probability $f_{i,j}^{(t)} - \lfloor f_{i,j}^{(t)} \rfloor$, and $\xi_{i,j}^{(t)} = \lfloor f_{i,j}^{(t)} \rfloor$ otherwise. This ensures the unbiasedness property $\mathbb{E}[\xi_{i,j}^{(t)}] = f_{i,j}^{(t)}$. The final quantized integer is given by $\bar{g}_{i,j}^{(t)} = \text{sgn}(g_{i,j}^{(t)}) \times \xi_{i,j}^{(t)}$. To resolve the conflict between limited bit-width and negative number representation in finite fields noted in prior works, we encode $\bar{g}_{i,j}^{(t)}$ using $w_i^{(t)}$ -bit Two's Complement representation. This allows us to strictly bound the transmission to $w_i^{(t)}$ bits without information loss, deferring the mapping to the computation field $\mathbb{Z}_p$ until the secure aggregation phase at the servers. The adaptive gradient compression protocol is formally detailed in Protocol 1.

Boolean Secret Sharing with Uniform Bit-Width: Following quantization, each client generates Boolean secret shares using a modified bit decomposition approach. In our original design, variable bit-lengths for different gradient components could reveal magnitude information. To address this security vulnerability, we enforce a *uniform bit-width* policy. Instead of determining precision component-wise, we assign a single bit-width $w_i^{(t)}$ for the entire gradient vector based on the client's current adaptive compression level $s_i^{(t)}$. Specifically, we set $w_i^{(t)} = \lceil \log_2(s_i^{(t)} + 1) \rceil + 1$ (including the sign bit). This ensures that all secret shares have identical lengths regardless of the underlying value's magnitude, effectively masking the data distribution. Following the integer quantization, each component $\bar{g}_{i,j}^{(t)}$ is converted to its $w_i^{(t)}$ -bit Two's Complement binary representation. This bit-string is then decomposed into $w_i^{(t)}$ Boolean shares: $\bar{g}_{i,j}^{(t)} = \bigoplus_{k=0}^{w_i^{(t)}-1} x_{i,j,k}^{(0)} \oplus x_{i,j,k}^{(1)}$. To prevent malicious clients from inflating $w_i^{(t)}$ arbitrarily, servers enforce a global ceiling $W_{\max}$ (set to 8 bits), rejecting any client whose declared bit-width exceeds it. The effective bit-width in any round is therefore $w_i^{(t,\text{eff})} = \min(\lceil \log_2(s_i^{(t)} + 1) \rceil + 1, W_{\max})$, and in practice falls within $[2, W_{\max}]$, yielding 4–6 bits per element across most training rounds.

Security Against Distribution and Sparsity Leakage: The uniform bit-width policy provides protection against two leakage. *Gradient distribution leakage.* Since all m components of client i are encoded at the same $w_i^{(t)}$ -bit width, no server can infer per-component magnitudes from share lengths. Each share $x_{i,j,k}^{(0)} \xleftarrow{\$} \{0, 1\}$ is uniformly random. The only observable to a single server is $w_i^{(t)}$, which encodes $s_i^{(t)}$ rather than any per-component statistic. *Gradient sparsity leakage.* Under $w_i^{(t)}$ -bit Two's Complement, zero-valued and non-zero components produce Boolean shares of identical length. After XOR masking, the share of a zero bit is statistically indistinguishable from that of a one bit. Moreover, stochastic quantization randomizes which small-magnitude components are mapped to zero, ensuring the post-quantization sparsity pattern does not reflect the original gradient structure.

Protocol Integration: The complete adaptive compression and Boolean sharing process is integrated in Protocol 2, which first performs compression operations and then applies Boolean secret sharing to the compressed results. This protocol maintains both the communication efficiency benefits of gradient compression and the strong security properties required for secure multi-party computation.

B. Client-Side Correlation Generation for Secure Differential Privacy

In distributed learning systems with DP requirements, secure computation protocols must support complex nonlinear operations including secure gradient clipping, reciprocal computation, and noise addition, which cannot be achieved through simple linear secret sharing alone. Existing approaches [23], [24] rely on expensive server-to-server interactive protocols for correlation generation, creating computational bottlenecks that

Protocol 2: Boolean Secret Sharing With Uniform Width.

Input: Quantized integers $\{\bar{g}_{i,j}^{(t)}\}_{j=1}^m$ of client i , max bit-width $W_{\max}$.

Output: Boolean shares $\{x_{i,j,k}^{(0)}\}_{k=0}^{w_i^{(t)}-1}, \{x_{i,j,k}^{(1)}\}_{k=0}^{w_i^{(t)}-1}$.

- 1: Execute Protocol 1 to obtain $\{\bar{g}_{i,j}^{(t)}\}_{j=1}^m$.
- 2: $w_i^{(t)} \leftarrow \lceil \log_2(s_i^{(t)} + 1) \rceil + 1$.
- 3: **Assert** $w_i^{(t)} \leq W_{\max}$.
- 4: **if** Check fails **then**
- 5: **Abort** protocol.
- 6: **end if**
- 7: **for** $j \in [m]$ **do**
- 8: Convert $\bar{g}_{i,j}^{(t)}$ to $w_i^{(t)}$ -bit Two's Complement representation:
- 9: $B_{i,j}^{(t)} \leftarrow \bar{g}_{i,j}^{(t)} \pmod{2^{w_i^{(t)}}}$.
- 10: Decompose $B_{i,j}^{(t)} = \sum_{k=0}^{w_i^{(t)}-1} 2^k \bar{g}_{i,j,k}^{(t)}$ where $\bar{g}_{i,j,k}^{(t)} \in \{0, 1\}$.
- 11: **for** $k = 0$ to $w_i^{(t)} - 1$ **do**
- 12: Sample $x_{i,j,k}^{(0)} \xleftarrow{\$} \{0, 1\}$.
- 13: Set $x_{i,j,k}^{(1)} \leftarrow \bar{g}_{i,j,k}^{(t)} \oplus x_{i,j,k}^{(0)}$.
- 14: **end for**
- 15: **end for**
- 16: Send $\{x_{i,j,k}^{(0)}\}, w_i^{(t)}$ to S_0 ; $\{x_{i,j,k}^{(1)}\}, w_i^{(t)}$ to S_1 .
- 17: Servers verify $w_i^{(t)} \leq W_{\max}$ and match received share count.
- 18: Reject client i if verification fails.

scale quadratically with clients and require specialized cryptographic primitives for operations like secure norm computation, reciprocal calculation, and conditional scaling within encrypted domains.

Our correlation generation framework is designed to satisfy three technical requirements. First, it must support efficient Boolean-to-arithmetic conversion to process compressed gradient representations. Second, it must enable ℓ_2 norm computation over secret-shared gradient vectors through secure squaring. Third, it must handle conditional operations, i.e., secure comparison and conditional assignment, for gradient clipping and noise scaling. Our approach develops a unified client-side correlation generation strategy where each client pre-computes and distributes correlation data for all required operations, transforming the correlation bottleneck from interactive server computation to efficient local client operations with batch verification.

Oblivious Transfer Correlations for Boolean Operations: To enable efficient Boolean-to-arithmetic conversion compatible with the secure aggregation field, we generate Δ -COT correlations directly over the arithmetic field $\mathbb{Z}_p$. Each client i creates correlation pairs (q_j, t_j) where $q_j \xleftarrow{\$} \mathbb{Z}_p$ and $t_j = q_j + x_j^{(1)} \cdot \Delta^{(i)} \pmod{p}$. Here, $\Delta^{(i)} \xleftarrow{\$} \mathbb{Z}_p$ serves as the client's global offset parameter, and $x_j^{(1)}$ represents Boolean gradient shares held by server S_1 mapped to $\mathbb{Z}_p$. This ensures

Protocol 3: OT Correlation Generation.

Input: Boolean shares $\{x_{i,j,k}^{(1)}\}_{j \in [m], k \in [w]}$ of client i , prime p .

Output: Δ -COT correlations $(Q^{(i)}, T^{(i)})$ in $\mathbb{Z}_p$.

- 1: Sample global offset: $\Delta^{(i)} \xleftarrow{\$} \mathbb{Z}_p$.
- 2: Flatten shares: $x^{(1)} \leftarrow \text{concatenate}(\{x_{i,j,k}^{(1)}\}_{j,k})$.
- 3: **for** $j = 1$ to n **do**
- 4: Sample $q_j \xleftarrow{\$} \mathbb{Z}_p$.
- 5: Compute $t_j \leftarrow q_j + x_j^{(1)} \cdot \Delta^{(i)} \pmod{p}$.
- 6: **end for**
- 7: Set $Q^{(i)} \leftarrow \{q_1, \dots, q_n\}, T^{(i)} \leftarrow \{t_1, \dots, t_n\}$.
- 8: Send $(Q^{(i)}, \Delta^{(i)})$ to server S_0 .
- 9: Send $T^{(i)}$ to server S_1 .

Protocol 4: Square Correlation Generation.

Input: Parameters m, u, κ .

Output: Square correlations $W^{(0)}, W^{(1)}$ distributed to servers.

- 1: Set $v \leftarrow u + \kappa + 3$.
- 2: Sample $\{\alpha_k\}_{k=1}^{2m} \xleftarrow{\$} \mathbb{Z}_{2^v}$.
- 3: **for** $k = 1$ to $2m$ **do**
- 4: Compute $\beta_k \leftarrow \alpha_k^2 \pmod{2^v}$.
- 5: **end for**
- 6: Generate arithmetic shares of $\{(\alpha_k, \beta_k)\}_{k=1}^{2m}$.
- 7: **for** $k = 1$ to $2m$ **do**
- 8: Sample $\alpha_k^{(0)} \xleftarrow{\$} \mathbb{Z}_{2^v}$, set $\alpha_k^{(1)} \leftarrow \alpha_k - \alpha_k^{(0)}$.
- 9: Sample $\beta_k^{(0)} \xleftarrow{\$} \mathbb{Z}_{2^v}$, set $\beta_k^{(1)} \leftarrow \beta_k - \beta_k^{(0)}$.
- 10: **end for**
- 11: Set $W^{(0)} \leftarrow \{(\alpha_k^{(0)}, \beta_k^{(0)})\}_{k=1}^{2m}$,
 $W^{(1)} \leftarrow \{(\alpha_k^{(1)}, \beta_k^{(1)})\}_{k=1}^{2m}$.
- 12: Send $W^{(0)}$ to server S_0 , send $W^{(1)}$ to server S_1 .

that the arithmetic reconstruction $z^{(b)} = \sum 2^k (x^{(b)} - 2y^{(b)})$ is computed consistently within the field $\mathbb{Z}_p$. This correlation generation creates tuples $\{(q_j, t_j)\}_{j=1}^n$ enabling the conversion protocol detailed in Protocol 3.

Square Correlations for ℓ_2 Norm Computation: DP protocols require computing ℓ_2 norms of gradient vectors within secret-shared domains, necessitating secure squaring operations for each gradient component. We develop specialized square correlations (α_k, β_k) where $\beta_k = \alpha_k^2 \pmod{2^u}$ that enable efficient sacrifice-based verification of quadratic computations. Each client i generates $2m$ correlation pairs to support both the required squaring operations and verification procedures by sampling $\{\alpha_k\}_{k=1}^{2m} \xleftarrow{\$} \mathbb{Z}_{2^v}$ where $v = u + \kappa + 3$ provides sufficient precision for verification, computing $\beta_k = \alpha_k^2 \pmod{2^v}$, and generating arithmetic secret shares of these correlation pairs, as formalized in Protocol 4.

Batch Verification for Correlation Correctness: To defend against malicious clients and collusion between a malicious server and client, we implement a robust verification mechanism using the Fiat-Shamir heuristic. Instead of allowing servers to arbitrarily pick challenge coefficients, the challenges are

Protocol 5: Robust Batch Verification.**Input:** Servers S_0, S_1 , correlations from all clients, prime p .**Output:** Verified correlations or abort.

- 1: **for** each client i **do**
- 2: Servers exchange hash commitments of received correlations.
- 3: Compute seed: $\mathcal{S} \leftarrow \text{Hash}(Q^{(i)} || T^{(i)} || \Delta^{(i)})$.
- 4: Generate challenges: $\{\chi_j\}_{j=1}^n \leftarrow \text{PRG}(\mathcal{S}) \in \mathbb{Z}_p^n$.
- 5: Server S_1 computes: $\tilde{x} \leftarrow \sum x_j^{(1)} \chi_j, \tilde{t} \leftarrow \sum t_j \chi_j \pmod{p}$.
- 6: Server S_0 computes: $\tilde{q} \leftarrow \sum q_j \chi_j \pmod{p}$.
- 7: Check: $\tilde{t} \stackrel{?}{=} \tilde{q} + \tilde{x} \cdot \Delta^{(i)} \pmod{p}$.
- 8: If check fails, reject client i and continue.
- 9: **for** each pair $(\alpha, \beta), (\hat{\alpha}, \hat{\beta})$ in $W^{(0)}, W^{(1)}$ **do**
- 10: Sample odd challenge $\tau \xleftarrow{\$} \mathbb{Z}_{2^v}$.
- 11: Open $e \leftarrow \tau\alpha - \hat{\alpha}$ to both servers.
- 12: Compute verification shares:
 $s^{(b)} \leftarrow \tau^2 \beta^{(b)} - \hat{\beta}^{(b)} - 2\tau e \alpha^{(b)} + e^2$.
- 13: Check: $s^{(0)} + s^{(1)} \stackrel{?}{=} 0 \pmod{2^v}$.
- 14: If check fails, reject client i and continue.
- 15: **end for**
- 16: **end for**

derived deterministically via a Random Oracle (RO) applied to the committed correlations. Servers first exchange commitments of their shares. Then, they compute the challenge seed $\mathcal{S} = \text{Hash}(Q^{(i)} || T^{(i)} || \Delta^{(i)})$. The verification coefficients $\{\chi_j\}_{j=1}^n$ are generated from this seed. This prevents a malicious server from pre-selecting challenges to help a malicious client pass verification. The check $\sum_{j=1}^n \chi_j t_j = \sum_{j=1}^n \chi_j q_j + (\sum_{j=1}^n \chi_j x_j^{(1)}) \cdot \Delta^{(i)}$ is then performed in $\mathbb{Z}_p$. Any mismatch or refusal to participate results in an identifiable abort, preserving safety, as detailed in Protocol 5.

Protocol Integration: The generated correlations integrate seamlessly with our secure aggregation process through a structured composition phase where Boolean shares are converted to arithmetic form using verified OT correlations through the relationship $z^{(b)} = \sum_{k=0}^{w-1} 2^k (x_{j,k}^{(b)} - 2y_{j,k}^{(b)})$ with $y_{j,k}^{(b)}$ representing arithmetic shares of the Boolean multiplication $x_{j,k}^{(0)} \wedge x_{j,k}^{(1)}$, while square correlations enable secure norm computation through verified squaring operations, supporting the complex DP mechanisms detailed in Protocol 6. This client-side correlation generation framework transforms the computational bottleneck from expensive server-to-server protocols to efficient local operations with batch verification, enabling practical deployment of secure DP in distributed learning systems while maintaining strong security guarantees against malicious clients.

C. Secure Differential Privacy Integration

While cryptographic privacy mechanisms protect computation processes, aggregated results may still leak information through membership inference attacks. DP addresses this by providing formal guarantees that individual participation has

bounded impact on outputs, but implementing DP within secure multi-party computation presents significant computational challenges for gradient clipping operations requiring nonlinear functions over secret-shared values.

The main computational requirement involves computing the reciprocal square root function $1/\|\langle \bar{g}^{(t)} \rangle\|_2$ needed for gradient clipping, where $\langle \bar{g}^{(t)} \rangle$ represents secret-shared aggregated gradients. This operation is challenging because nonlinear functions like division and square root are computationally expensive in secure multi-party computation, requiring precise approximation methods that maintain accuracy while preserving security throughout multi-step computations. Fortress avoids this difficulty through polynomial approximation. The reciprocal square root is decomposed into two phases: a normalization step and a polynomial evaluation step. Both phases operate directly over secret-shared values, preserving security without sacrificing approximation accuracy. Fortress employs three integrated phases: reciprocal square root computation through normalization and polynomial approximation, secure gradient clipping using computed reciprocal norms with conditional operations, and calibrated noise addition to achieve formal DP guarantees. This approach enables efficient implementation of complex DP operations while maintaining security throughout the process, as formalized in Protocol 6.

Gradient clipping requires computing $1/\|\bar{g}^{(t)}\|_2$, a nonlinear operation that is prohibitively expensive in secret-shared domains. The key enabler is dynamic normalization: by extracting the bit-length of the squared norm via a prefix-sum circuit, the input is mapped to a fixed bounded interval regardless of gradient magnitude. On this compact domain, a low-degree Chebyshev polynomial approximates the reciprocal square root with negligible error relative to the DP noise, and requires only additions and multiplications, fitting directly into the Fortress.

Reciprocal Square Root Computation via Polynomial Approximation: The first phase implements reciprocal square root computation through dynamic normalization. The input $x = \|\langle \bar{g}^{(t)} \rangle\|_2^2$ is unbounded, so we first determine its magnitude scale. We define e not as a single bit, but as the *integer bit-length* of the squared norm, i.e., $e = \lfloor \log_2 x \rfloor + 1$. This is computed via a prefix-sum circuit on the binary representation of x to find the position of the most significant “1”. Servers then compute the normalized value $\hat{x} = x \cdot 2^{-(e+1)}$ using secure truncation. Since $2^{e-1} \leq x < 2^e$, it follows that $0.25 \leq \hat{x} < 0.5$, strictly satisfying the convergence domain required for our Chebyshev approximation. We then evaluate $f(\hat{x}) \approx 1/\sqrt{\hat{x}}$ and scale back the result by computing $1/\sqrt{x} \approx f(\hat{x}) \cdot 2^{-(e+1)/2}$.

For polynomial approximation of $f(\hat{x}) = 1/\sqrt{\hat{x}}$ on $[0.25, 0.5]$, we use degree-4 Chebyshev approximation coefficients adapted for fixed-point arithmetic in secure computation. The degree-4 choice balances approximation accuracy with computational efficiency: higher degrees provide marginal accuracy improvements while significantly increasing secure multiplication rounds, while lower degrees fail to meet error requirement for DP precision. The coefficients are computed through the following systematic process: First, transform the interval $[0.25, 0.5]$ to the standard Chebyshev interval $[-1, 1]$ using the

linear mapping $t = \frac{2\hat{x}-0.75}{0.25} = 8\hat{x} - 3$, which transforms the function to $g(t) = ((t+3)/8)^{-1/2}$. Second, compute the Chebyshev coefficients using the orthogonality relation $c_k = \frac{2}{\pi} \int_{-1}^1 \frac{g(t)T_k(t)}{\sqrt{1-t^2}} dt$ where $T_k(t)$ are Chebyshev polynomials of the first kind. These integrals are evaluated numerically using 64-point Gauss-Chebyshev quadrature, yielding the coefficients in the transformed domain: $c_0 = 2.922$, $c_1 = -5.435$, $c_2 = 6.735$, $c_3 = -4.358$, $c_4 = 1.137$. Third, transform back to the original interval by substituting $t = 8\hat{x} - 3$ into the Chebyshev expansion $\sum_{k=0}^4 c_k T_k(t)$, yielding the polynomial $p(\hat{x}) = 1.137\hat{x}^4 - 4.358\hat{x}^3 + 6.735\hat{x}^2 - 5.435\hat{x} + 2.922$.

The approximation error is bounded by $|f(\hat{x}) - p(\hat{x})| \leq \frac{(b-a)^5}{2^{20} \cdot 5!} \max |f^{(5)}(\xi)| = \frac{(0.25)^5}{2^{20} \cdot 120} \cdot 62 < 2^{-40}$ using the standard Chebyshev error bound with $f^{(5)}(x) = -\frac{945}{32}x^{-11/2}$.

Secure Gradient Clipping with Conditional Operations: The second phase implements secure gradient clipping by combining the computed reciprocal norm with clipping bound C to realize the DP operation $\bar{g}_{\text{clipped}} = \bar{g} \times \min(1, C/\|\bar{g}\|_2)$ entirely within the secret-shared domain. The clipping factor is computed as $\langle \alpha \rangle^{(b)} = C \times \langle 1/\|\bar{g}^{(t)}\|_2 \rangle^{(b)}$ through secure multiplication, followed by the SecureCompare operation that determines whether clipping is necessary by comparing $\langle \alpha \rangle$ with 1. The SecureCompare operation implements secure comparison by computing the difference $\langle \delta \rangle^{(b)} = \langle \alpha \rangle^{(b)} - 1$, extracting the sign bit through arithmetic-to-binary conversion followed by most significant bit extraction, specifically by converting $\langle \delta \rangle$ to binary representation $\sum_{i=0}^{u-1} 2^i \langle d_i \rangle$ and extracting $\langle d_{u-1} \rangle$ as the sign bit, outputting Boolean indicator $b = \langle d_{u-1} \rangle \in \{0, 1\}$ where $b = 1$ indicates $\alpha > 1$ (clipping required) and $b = 0$ indicates $\alpha \leq 1$ (no clipping needed) [31]. The conditional scaling applies clipping through secure multiplexing: $\langle \bar{g}_{\text{clipped}}^{(t)} \rangle^{(b)} = (1-b)\langle \bar{g}^{(t)} \rangle^{(b)} + b \times \langle \alpha \rangle^{(b)} \times \langle \bar{g}^{(t)} \rangle^{(b)}$, preserving original gradients when $b = 0$ and applying scaling when $b = 1$, ensuring clipped gradients satisfy $\|\bar{g}_{\text{clipped}}\|_2 \leq C$ as required for DP.

Calibrated Noise Addition for Privacy Protection: The final phase provides calibrated privacy protection through coordinated noise addition that maintains required statistical properties while preserving security throughout the process. Each server independently samples Gaussian noise $n^{(t),(s)} \sim \mathcal{N}(0, \sigma^2 I_m)$ where noise parameter $\sigma = \sqrt{2 \ln(1.25/\delta)} \cdot C/\epsilon$ is determined by privacy parameters (ϵ, δ) and clipping bound C according to theoretical requirements of the Gaussian mechanism for achieving (ϵ, δ) -DP. The servers coordinate through additive secret sharing where server S_0 generates arithmetic shares of $n^{(t),(0)}$ by sampling $n^{(t),(0),0} \xleftarrow{\$} \mathcal{N}(0, \sigma^2 I_m)$, setting $n^{(t),(0),1} = n^{(t),(0)} - n^{(t),(0),0}$, and sending $n^{(t),(0),1}$ to server S_1 , while server S_1 similarly shares $n^{(t),(1)}$ with server S_0 , resulting in combined noise $\langle n^{(t)} \rangle^{(b)} = n^{(t),(0)} + n^{(t),(1)}$ that maintains the required Gaussian distribution with variance $2\sigma^2 I_m$ while remaining secret-shared throughout the computation. The protocol concludes by computing final DP gradients as $\langle \tilde{g}^{(t)} \rangle^{(b)} = \eta/B \cdot (\langle \bar{g}_{\text{clipped}}^{(t)} \rangle^{(b)} + \langle n^{(t)} \rangle^{(b)})$, incorporating both clipped gradients and calibrated noise while scaling appropriately with learning rate η and batch size B for distributed learning optimization.

Protocol 6: Secure Differential Privacy Integration.

Input: Aggregated shares $\langle \bar{g}^{(t)} \rangle^{(b)}$, $\langle \|\bar{g}^{(t)}\|_2^2 \rangle^{(b)}$, clipping bound C , privacy parameters (ϵ, δ) , learning rate η , batch size B .

Output: DP gradient shares $\langle \tilde{g}^{(t)} \rangle^{(0)}, \langle \tilde{g}^{(t)} \rangle^{(1)}$.

- 1: $e \leftarrow \text{SecureBitLength}(\langle \|\bar{g}^{(t)}\|_2^2 \rangle^{(b)})$.
 - 2: $\langle \hat{x} \rangle^{(b)} \leftarrow \langle \|\bar{g}^{(t)}\|_2^2 \rangle^{(b)} \cdot 2^{-(e+1)}$.
 - 3: Compute polynomial powers using square correlations from Protocol 4:
 - 4: $\langle \hat{x}^2 \rangle^{(b)} \leftarrow \text{secure squaring}$,
 $\langle \hat{x}^3 \rangle^{(b)} \leftarrow \langle \hat{x} \rangle^{(b)} \times \langle \hat{x}^2 \rangle^{(b)}$.
 - 5: $\langle \hat{x}^4 \rangle^{(b)} \leftarrow \langle \hat{x}^2 \rangle^{(b)} \times \langle \hat{x}^2 \rangle^{(b)}$.
 - 6: $\langle f(\hat{x}) \rangle^{(b)} \leftarrow 1.137\langle \hat{x}^4 \rangle^{(b)} - 4.358\langle \hat{x}^3 \rangle^{(b)} + 6.735\langle \hat{x}^2 \rangle^{(b)} - 5.435\langle \hat{x} \rangle^{(b)} + 2.922$.
 - 7: $\langle 2^{-e/2} \rangle^{(b)} \leftarrow \text{SecureExponentiation}(e)$.
 - 8: $\langle 1/\|\bar{g}^{(t)}\|_2 \rangle^{(b)} \leftarrow \langle f(\hat{x}) \rangle^{(b)} \times \langle 2^{-e/2} \rangle^{(b)}$.
 - 9: $\langle \alpha \rangle^{(b)} \leftarrow C \times \langle 1/\|\bar{g}^{(t)}\|_2 \rangle^{(b)}$.
 - 10: $b \leftarrow \text{SecureCompare}(\langle \alpha \rangle^{(b)}, 1)$.
 - 11: $\langle \bar{g}_{\text{clipped}}^{(t)} \rangle^{(b)} \leftarrow$
 $(1-b)\langle \bar{g}^{(t)} \rangle^{(b)} + b \times \langle \alpha \rangle^{(b)} \times \langle \bar{g}^{(t)} \rangle^{(b)}$.
 - 12: $\sigma \leftarrow \sqrt{2 \ln(1.25/\delta)} \cdot C/\epsilon$.
 - 13: Each server s samples: $n^{(t),(s)} \sim \mathcal{N}(0, \frac{\sigma^2}{2} I_m)$.
 - 14: Server S_0 shares $n^{(t),(0)}$ with S_1 , Server S_1 shares $n^{(t),(1)}$ with S_0 .
 - 15: $\langle n^{(t)} \rangle^{(b)} \leftarrow n^{(t),(0)} + n^{(t),(1)}$ (distributed $\mathcal{N}(0, \sigma^2 I_m)$).
 - 16: Compute $\langle \|n^{(t)}\|_2^2 \rangle^{(b)}$ using Square Correlations.
 - 17: Open $N_{sq} \leftarrow \text{Reconstruct}(\langle \|n^{(t)}\|_2^2 \rangle^{(b)})$.
 - 18: **Assert** $|N_{sq} - m \cdot (\sigma^2)| \leq \text{Threshold}$.
 - 19: **if** Check fails **then**
 - 20: **Abort**.
 - 21: **end if**
 - 22: $\langle \tilde{g}^{(t)} \rangle^{(b)} \leftarrow \frac{\eta}{B} (\langle \bar{g}_{\text{clipped}}^{(t)} \rangle^{(b)} + \langle n^{(t)} \rangle^{(b)})$.
-

Verifiable Calibrated Noise Addition: To address the threat where a malicious server injects arbitrary noise to disrupt training or compromise privacy, we enhance the noise generation with a lightweight verification step. Servers generate Gaussian noise shares $n^{(t),(b)}$ and, before addition, secure verify their validity using the pre-computed Square Correlations. Specifically, servers securely compute the squared ℓ_2 norm of the noise vector $\langle \|n^{(t)}\|_2^2 \rangle$. Since the sum of squares of Gaussian variables follows a Chi-squared distribution, honest noise must satisfy statistical bounds with high probability. We enforce a check $\langle \|n^{(t)}\|_2^2 \rangle \in [\mu_{\chi^2} - \kappa\sigma_{\chi^2}, \mu_{\chi^2} + \kappa\sigma_{\chi^2}]$. Any server attempting to inject outlier noise will trigger a protocol abort.

D. Malicious Security Via Transcript Emulation

The integration of adaptive gradient compression, client-side correlation generation, and secure DP operations into a unified protocol naturally combines these components into a cohesive framework. Our unified protocol employs a four-phase design where each phase builds upon previous outputs while

Protocol 7: Fortress.

Input: Clients $\mathcal{C}$, servers S_0, S_1 , model $\theta^{(t-1)}$, privacy parameters (ϵ, δ) , clipping bound C , model dimension m (size of parameter vector θ), pre-distributed keys and seeds (established in Fortress Bootstrapping).

Output: Updated model $\theta^{(t)}$ with privacy guarantees.

// Phase 1: Compression and Correlation Generation:

- 1: **for** each client $i \in \mathcal{C}$ **do**
- 2: Execute adaptive gradient compression with level $s_i^{(t)}$.
- 3: Generate Boolean secret shares and correlations via Protocols 3, 4.
- 4: Transmit shares and correlations to servers S_0, S_1 .
- 5: **end for**

// Phase 2: Verification and Aggregation:

- 6: Execute Protocol 5 for correlation verification.
- 7: Perform bit composition using verified correlations.
- 8: Compute aggregated shares $\langle \tilde{g}^{(t)} \rangle^{(b)}$ and $\langle \|\tilde{g}^{(t)}\|_2^2 \rangle^{(b)}$.

// Phase 3: Secure DP Operations:

- 9: Execute Protocol 6 for clipping and noise addition.
- 10: Generate DP gradient shares $\langle \tilde{g}^{(t)} \rangle^{(b)}$ with (ϵ, δ) -DP.

// Phase 4: Reconstruction and Update:

- 11: Reconstruct $\tilde{g}^{(t)} = \langle \tilde{g}^{(t)} \rangle^{(0)} + \langle \tilde{g}^{(t)} \rangle^{(1)}$.
- 12: Update model $\theta^{(t)} \leftarrow \theta^{(t-1)} - \tilde{g}^{(t)}$.
- 13: **return** $\theta^{(t)}$.

maintaining distinct computational and security properties, as shown in Protocol 7. However, distributed systems often require protection against malicious servers that deviate from protocol execution to compromise privacy or correctness. Traditional malicious security approaches using zero-knowledge proofs or fully homomorphic encryption incur prohibitive computational overhead for practical deployment [32], [33], [34]. The transcript emulation mechanism provides an efficient alternative that achieves malicious security with minimal overhead by leveraging the deterministic nature of secure computation protocols, enabling clients to predict and verify server behavior without expensive cryptographic primitives [24].

Extending malicious security guarantees requires addressing the predictability of server computations across all protocol phases. The main requirement lies in enabling clients to generate accurate transcripts of expected server messages while handling complex operations including reciprocal square root computation, secure comparison operations, and conditional scaling for DP. Fortress maintains deterministic message prediction capabilities by exploiting the fact that clients possess complete knowledge of their inputs, generated correlations, and the deterministic computation graph executed by servers. Our malicious security framework employs three integrated components: transcript prediction that generates expected message sequences for all protocol phases, cryptographic digest verification that provides efficient comparison through collision-resistant hashing, and abort-on-detection that identifies malicious behavior before sensitive information disclosure. This design ensures comprehensive protection against

Protocol 8: Malicious Security via Transcript Emulation.

Input: Client i with gradient shares $\tilde{g}_i^{(t)}$, correlations ρ_i , verification challenges χ .

Output: Transcript digest H_i for malicious detection.

- 1: Initialize transcript $T_i \leftarrow \emptyset$.
- 2: Simulate Phase 1: correlation verification messages using ρ_i and χ .
- 3: Simulate Phase 2: secure aggregation messages using known shares and correlations.
- 4: Simulate Phase 3: DP computation messages for polynomial evaluation and comparison.
- 5: Simulate Phase 4: reconstruction messages using predicted aggregated values.
- 6: Exclude verification results: $T_i^* \leftarrow T_i \setminus \mathcal{R}_i$.
- 7: $H_i \leftarrow \text{Hash}(T_i^*)$.
- 8: Send H_i to both servers S_0, S_1 .
- 9: Servers compute hash $H_s \leftarrow \text{Hash}(\mathcal{M} \setminus \mathcal{R})$.
- 10: Before final output: Check $H_i \stackrel{?}{=} H_s$ for all clients i .
- 11: If any mismatch detected: abort protocol before output revelation.
- 12: Else: proceed with model update.

malicious servers while maintaining the efficiency benefits of client-generated correlations, as formalized in Protocol 8.

Client-Side Transcript Prediction: The mechanism relies on the deterministic nature of secure computation protocols when viewed from the client's perspective. Each client maintains complete knowledge of the deterministic computation graph executed on their input, enabling accurate transcript generation across all protocol phases. The transcript function $\mathcal{T}_i(\tilde{g}_i^{(t)}, \rho_i, \chi) \rightarrow T_i$ generates the complete sequence of messages that servers exchange during processing of client i 's contribution, where $\tilde{g}_i^{(t)}$ represents client's gradient shares, ρ_i denotes generated correlations, and χ captures verification challenges revealed after the first phase. This prediction capability extends naturally to our protocol because clients know their inputs, the correlations they generate, and can simulate the secure computation operations that servers will perform.

The transcript generation operates across all four phases of our unified protocol. For Phase 1 correlation verification, client i simulates batch OT verification by predicting linear combinations $\tilde{t} = \sum_{j=1}^n t_j \cdot \chi_j$, $\tilde{q} = \sum_{j=1}^n q_j \cdot \chi_j$, and $\tilde{x} = \sum_{j=1}^n x_j^{(1)} \cdot \chi_j$ that servers compute, along with sacrifice-based square correlation verification messages $\tau^2\beta - \hat{\beta} - 2\tau\epsilon\alpha + \epsilon^2$ for each correlation pair. For Phase 2 secure aggregation, clients simulate bit multiplication results $y_{j,k}^{(b)} = x_{j,k}^{(0)} \wedge x_{j,k}^{(1)}$ computed using OT correlations and arithmetic conversion $z_j^{(b)} = \sum_{k=0}^{w-1} 2^k (x_{j,k}^{(b)} - 2y_{j,k}^{(b)})$ for Boolean-to-arithmetic transformation. For Phase 3 DP operations, clients generate transcripts for reciprocal square root computation, polynomial approximation, and secure comparison operations by simulating computations with representative values that maintain identical message structures without revealing actual values.

Digest-Based Verification and Detection: The verification mechanism employs cryptographic digests to minimize communication overhead while ensuring malicious behavior detection before privacy compromise. Each client computes a collision-resistant hash of their predicted transcript, excluding intermediate verification results: $H_i = \text{Hash}(T_i \setminus \mathcal{R}_i)$, while servers incrementally compute hashes over their actual computation steps $H_s = \text{Hash}(\mathcal{M} \setminus \mathcal{R})$, where $\mathcal{M}$ represents actual server messages and $\mathcal{R}$ denotes verification results. The protocol execution follows a specific sequence where clients submit correlations and gradient shares in Phase 1 without knowledge of verification challenges, maintaining correlation verification soundness. After servers reveal verification challenges, clients complete transcript prediction and submit verification digests. Servers proceed with Phases 2–4 while maintaining running hashes of their actual message exchanges. Before revealing final results, servers compare observed transcript hashes with client digests, with any mismatch $H_i \neq H_s$ indicating malicious tampering and causing the honest server to abort before sensitive information disclosure.

IV. THEORETICAL ANALYSIS

A. Adaptive Gradient Compression

Lemma 1 (Boolean Secret Sharing Correctness): Protocol 2 correctly generates Boolean secret shares of compressed gradient components. For any quantized gradient component $\bar{g}_{i,j}^{(t)}$ with adaptive bit-width $w_i^{(t)}$, the reconstruction $\bar{g}_{i,j}^{(t)} = \bigoplus_{k=0}^{w_i^{(t)}-1} x_{i,j,k}^{(0)} \oplus x_{i,j,k}^{(1)}$ holds with probability 1.

Proof: The correctness follows from the construction in Protocol 2 and the properties of XOR operations over $\mathbb{F}_2$. For each quantized gradient component $\bar{g}_{i,j}^{(t)}$, the bit decomposition $\bar{g}_{i,j}^{(t)} = \sum_{k=0}^{w_i^{(t)}-1} 2^k \bar{g}_{i,j,k}^{(t)}$ where $\bar{g}_{i,j,k}^{(t)} \in \{0, 1\}$ is a standard binary representation. The bit-width $w_i^{(t)} = \lceil \log_2(\max_j(|\bar{g}_{i,j}^{(t)}|) + 1) \rceil$ ensures sufficient precision to represent all gradient components without overflow.

For each bit $k \in [0, w_i^{(t)} - 1]$, the client samples $x_{i,j,k}^{(0)} \xleftarrow{\$} \{0, 1\}$ uniformly at random from the binary field $\mathbb{F}_2$ and computes the complementary share as $x_{i,j,k}^{(1)} = \bar{g}_{i,j,k}^{(t)} \oplus x_{i,j,k}^{(0)}$. By the definition of XOR operation in $\mathbb{F}_2$, we have $x_{i,j,k}^{(0)} \oplus x_{i,j,k}^{(1)} = x_{i,j,k}^{(0)} \oplus (\bar{g}_{i,j,k}^{(t)} \oplus x_{i,j,k}^{(0)}) = \bar{g}_{i,j,k}^{(t)}$, where the second equality follows from the associativity and self-inverse properties of XOR. The full gradient component is recovered as $\bar{g}_{i,j}^{(t)} = \sum_{k=0}^{w_i^{(t)}-1} 2^k \bar{g}_{i,j,k}^{(t)} = \sum_{k=0}^{w_i^{(t)}-1} 2^k (x_{i,j,k}^{(0)} \oplus x_{i,j,k}^{(1)})$. The verification step ensures consistency by checking that the number of received bit shares matches the declared bit-width $w_i^{(t)}$ and validating component-wise bounds $\sum_{k=0}^{w_i^{(t)}-1} 2^k \leq 2^{w_i^{(t)}} - 1$, preventing overflow and ensuring the binary representation is well-formed. Since each step preserves correctness deterministically, reconstruction succeeds with probability 1. $\square$

Lemma 2 (Compression Unbiasedness): The adaptive compression mechanism in Protocol 1 preserves expected values. For any gradient component $g_{i,j}^{(t)}$, we have $\mathbb{E}[\hat{g}_{i,j}^{(t)}] = g_{i,j}^{(t)}$.

Proof: The compressed gradient component is computed as $\hat{g}_{i,j}^{(t)} = \|g_i^{(t)}\|_2 \times \text{sgn}(g_{i,j}^{(t)}) \times \xi(g_{i,j}^{(t)}, s_i^{(t)})$ where $\xi(g_{i,j}^{(t)}, s_i^{(t)})$ is the random quantization variable that depends on the compression level $s_i^{(t)}$ and the relative magnitude of the gradient component. The component-wise compression level is determined by $f_{i,j}^{(t)} = \lfloor s_i^{(t)} \times |g_{i,j}^{(t)}| / \|g_i^{(t)}\|_2 \rfloor$, which ensures that gradient components with larger relative magnitudes receive higher precision quantization levels, preserving the most significant directional information while aggressively compressing less important components. The random quantization variable $\xi(g_{i,j}^{(t)}, s_i^{(t)})$ is constructed to satisfy the unbiasedness $\mathbb{E}[\xi(g_{i,j}^{(t)}, s_i^{(t)})] = |g_{i,j}^{(t)}| / \|g_i^{(t)}\|_2$, which follows from unbiased quantization where the quantization noise is zero-mean.

Taking the expectation of the compressed gradient: $\mathbb{E}[\hat{g}_{i,j}^{(t)}] = \mathbb{E}[\|g_i^{(t)}\|_2 \times \text{sgn}(g_{i,j}^{(t)}) \times \xi(g_{i,j}^{(t)}, s_i^{(t)})] = \|g_i^{(t)}\|_2 \times \text{sgn}(g_{i,j}^{(t)}) \times \mathbb{E}[\xi(g_{i,j}^{(t)}, s_i^{(t)})] = \|g_i^{(t)}\|_2 \times \text{sgn}(g_{i,j}^{(t)}) \times |g_{i,j}^{(t)}| / \|g_i^{(t)}\|_2 = \text{sgn}(g_{i,j}^{(t)}) \times |g_{i,j}^{(t)}| = g_{i,j}^{(t)}$, where the second equality uses the linearity of expectation and the fact that $\|g_i^{(t)}\|_2$ and $\text{sgn}(g_{i,j}^{(t)})$ are deterministic given the input. The unbiasedness holds regardless of the adaptive compression level $s_i^{(t)}$ because the quantization is designed to preserve expected values at all compression levels, ensuring convergence properties are maintained throughout the adaptive process. $\square$

B. Correlation Generation Verification

Lemma 3 (OT Correlation Verification Soundness): If the OT correlation verification in Protocol 5 succeeds, the Δ -COT correlations are correct except with probability $2^{-\lambda}$.

Proof: The verification samples random coefficients $\{\chi_j\}_{j=1}^n \xleftarrow{\$} \mathbb{Z}_{2^\lambda}$ after correlations are received, where $n = m \times w_{avg}$ represents the total number of Boolean shares across all gradient components. For honestly generated correlations, each tuple (q_j, t_j) satisfies the Δ -COT relationship $t_j = q_j + x_j^{(1)} \cdot \Delta^{(i)} \pmod{2^\lambda}$ where $x_j^{(1)}$ is the Boolean gradient share held by server $\mathcal{S}_1$, $\Delta^{(i)}$ is client i 's global offset parameter, and all operations are performed in the ring $\mathbb{Z}_{2^\lambda}$. The verification check computes $\tilde{t} = \sum_{j=1}^n \chi_j t_j$, $\tilde{q} = \sum_{j=1}^n \chi_j q_j$, $\tilde{x} = \sum_{j=1}^n \chi_j x_j^{(1)}$ and verifies the relationship $\tilde{t} \stackrel{?}{=} \tilde{q} + \tilde{x} \cdot \Delta^{(i)} \pmod{2^\lambda}$.

Suppose a malicious client provides incorrect correlations with errors $\{e_j\}_{j=1}^n$ such that $t_j = q_j + x_j^{(1)} \cdot \Delta^{(i)} + e_j$ for at least one non-zero error term $e_j \neq 0$. The verification equation becomes $\tilde{t} - \tilde{q} - \tilde{x} \cdot \Delta^{(i)} = \sum_{j=1}^n \chi_j e_j$. Since at least one $e_j \neq 0$ and the challenge coefficients $\{\chi_j\}$ are uniformly and independently sampled from $\mathbb{Z}_{2^\lambda}$ after error commitment, let $E = \{j : e_j \neq 0\}$ be the set of positions with errors. For any fixed non-empty set E and error values $\{e_j\}_{j \in E}$, there exists at least one $j^* \in E$ such that e_{j^*} has a non-zero component that is coprime to 2^λ . The random coefficient χ_{j^*} then determines the distribution of the entire sum $\sum_{j=1}^n \chi_j e_j$, making it uniformly distributed over $\mathbb{Z}_{2^\lambda}$. Therefore, $\Pr[\sum_{j=1}^n \chi_j e_j = 0] = 1/2^\lambda$, giving the detection probability $1 - 2^{-\lambda}$. $\square$

Lemma 4 (Correlation Verification Soundness): If the square correlation verification succeeds, the selected correlations (α_k, β_k) satisfy $\beta_k = \alpha_k^2 \bmod 2^u$ except with probability $2^{-\kappa}$.

Proof: Consider two correlation pairs (α, β) and $(\hat{\alpha}, \hat{\beta})$ where potentially $\beta = \alpha^2 + \delta$ and $\hat{\beta} = \hat{\alpha}^2 + \delta'$ for error terms δ, δ' . The sacrifice verification uses a random odd challenge $\tau \xleftarrow{\$} \mathbb{Z}_{2^v}$ and computes $e = \tau\alpha - \hat{\alpha}$, then verifies $\tau^2\beta - \hat{\beta} - 2\tau e\alpha + e^2 \stackrel{?}{=} 0 \pmod{2^v}$. Expanding this equation yields $\tau^2\beta - \hat{\beta} - 2\tau(\tau\alpha - \hat{\alpha})\alpha + (\tau\alpha - \hat{\alpha})^2 = \tau^2(\beta - \alpha^2) - (\hat{\beta} - \hat{\alpha}^2) = \tau^2\delta - \delta'$. For the verification to pass with an incorrect primary correlation ($\delta \neq 0 \bmod 2^u$), we need $\tau^2\delta \equiv \delta' \pmod{2^v}$.

Let $\delta = 2^a \cdot \delta_{\text{odd}}$ where δ_{odd} is odd and $a < u$ (since $\delta \not\equiv 0 \pmod{2^u}$). Then $\tau^2 \cdot 2^a \cdot \delta_{\text{odd}} \equiv \delta' \pmod{2^v}$. Since τ is uniformly sampled from odd elements in $\mathbb{Z}_{2^v}$, write $\tau = 2k + 1$, then $\tau^2 = 4k^2 + 4k + 1 = 4k(k + 1) + 1$. Since exactly one of k or $k + 1$ is even, $4k(k + 1) \equiv 0 \pmod{8}$, so $\tau^2 \equiv 1 \pmod{8}$. When τ ranges over all odd elements in $\mathbb{Z}_{2^v}$, the values $\tau^2 \bmod 2^{v-a}$ have min-entropy at least $v - a - 3$ bits for $a < u$ and $v = u + \kappa + 3$. Since $v - a - 3 > \kappa$, a malicious client must predict the value of $\tau^2 \bmod 2^{v-a}$ before receiving τ , with success probability bounded by $2^{-\kappa}$. $\square$

C. Secure Differential Privacy Operations

Lemma 5 (Reciprocal Square Root Approximation Accuracy): The secure reciprocal square root computation in Protocol 6 achieves approximation error bounded by $\epsilon_{\text{approx}} \leq 2^{-40}$ for normalized inputs $\hat{x} \in [0.25, 0.5]$.

Proof: The input $\|\langle \bar{g}^{(t)} \rangle\|_2^2$ is normalized to $\hat{x} \times 2^e$ where e is determined by $\text{SecureMSB}(\|\langle \bar{g}^{(t)} \rangle\|_2^2)^{(b)}$ and $\hat{x} = \|\langle \bar{g}^{(t)} \rangle\|_2^2 / 2^e$ satisfies $0.25 \leq \hat{x} \leq 0.5$. The polynomial $p(\hat{x}) = 1.137\hat{x}^4 - 4.358\hat{x}^3 + 6.735\hat{x}^2 - 5.435\hat{x} + 2.922$ approximates $f(\hat{x}) = 1/\sqrt{\hat{x}}$ on this interval. By Chebyshev approximation theory [35], for the function $f(x) = x^{-1/2}$ on the interval $[0.25, 0.5]$, the optimal degree-4 polynomial minimizes the maximum approximation error $\|f - p_4\|_\infty = \min_{p \in \mathcal{P}_4} \max_{x \in [0.25, 0.5]} |f(x) - p(x)|$.

The fifth derivative satisfies $f^{(5)}(x) = -\frac{945}{32}x^{-11/2}$, giving $|f^{(5)}(x)| \leq \frac{945}{32} \cdot (0.25)^{-11/2} = \frac{945}{32} \cdot 2^{11} \approx 62$ for $x \in [0.25, 0.5]$. Using the error bound for Chebyshev approximation: $|f(x) - p_4(x)| \leq \frac{|0.5 - 0.25|^5}{2^{20} \cdot 5!} \max_{x \in [0.25, 0.5]} |f^{(5)}(x)| = \frac{(0.25)^5}{2^{20} \cdot 120} \cdot 62 \leq \frac{2^{-10}}{2^{20} \cdot 120} \cdot 62 < 2^{-40}$. The approximation is computed using verified square correlations from Protocol 4, ensuring that no additional error is introduced during secure polynomial evaluation. Therefore, $\epsilon_{\text{approx}} \leq 2^{-40}$. $\square$

Lemma 6 (Secure Gradient Clipping Correctness): The secure gradient clipping operation ensures $\|\bar{g}_{\text{clipped}}^{(t)}\|_2 \leq C$ while preserving gradient direction when $\|\bar{g}^{(t)}\|_2 \leq C$.

Proof: The clipping factor is computed as $\langle \alpha \rangle^{(b)} = C \times \langle 1/\|\bar{g}^{(t)}\|_2 \rangle^{(b)}$ where the reciprocal square root uses the polynomial approximation from Lemma 5. The $\text{SecureCompare}(\langle \alpha \rangle^{(b)}, 1)$ operation determines the Boolean indicator $b = 1$ if $\alpha > 1$ (equivalently, $\|\bar{g}^{(t)}\|_2 > C$) and $b = 0$ if $\alpha \leq 1$ (equivalently, $\|\bar{g}^{(t)}\|_2 \leq C$), implemented by computing

$\langle \delta \rangle^{(b)} = \langle \alpha \rangle^{(b)} - 1$, converting to binary representation, and extracting the sign bit. The conditional scaling operation $\langle \bar{g}_{\text{clipped}}^{(t)} \rangle^{(b)} = (1 - b)\langle \bar{g}^{(t)} \rangle^{(b)} + b \times \langle \alpha \rangle^{(b)} \times \langle \bar{g}^{(t)} \rangle^{(b)}$ implements a secure multiplexer between two cases.

When $b = 0$ (no clipping required): $\bar{g}_{\text{clipped}}^{(t)} = \bar{g}^{(t)}$, preserving both magnitude and direction with $\|\bar{g}_{\text{clipped}}^{(t)}\|_2 = \|\bar{g}^{(t)}\|_2 \leq C$. When $b = 1$ (clipping required): $\bar{g}_{\text{clipped}}^{(t)} = \alpha \cdot \bar{g}^{(t)} = (C/\|\bar{g}^{(t)}\|_2) \cdot \bar{g}^{(t)}$, yielding $\|\bar{g}_{\text{clipped}}^{(t)}\|_2 = (C/\|\bar{g}^{(t)}\|_2) \cdot \|\bar{g}^{(t)}\|_2 = C$ while preserving direction since $\alpha > 0$. The polynomial approximation error $\epsilon_{\text{approx}} \leq 2^{-40}$ introduces negligible deviation, with the actual clipped norm satisfying $C(1 - 2^{-40}) \leq \|\bar{g}_{\text{clipped}}^{(t)}\|_2 \leq C(1 + 2^{-40})$, effectively maintaining the clipping bound for practical purposes. $\square$

D. Semi-Honest Security Proofs

We define the ideal functionality for privacy-preserving distributed learning with compression and DP as $\mathcal{F}_{\text{Fortress}}$, which operates as follows: (1) Receive gradient vectors $\{g_i^{(t)}\}_{i \in \mathcal{C}_t}$ from participating clients in round t . (2) Apply adaptive compression with levels $\{s_i^{(t)}\}$ to obtain $\{\hat{g}_i^{(t)}\}$. (3) Enforce ℓ_2 bounds: reject gradients with $\|\hat{g}_i^{(t)}\|_2 > \mu$. (4) Compute aggregate $\bar{g}^{(t)} = \frac{1}{|\mathcal{C}_t|} \sum_{i \text{ accepted}} \hat{g}_i^{(t)}$. (5) Apply DP: $\tilde{g}^{(t)} = \frac{\eta}{B}(\text{Clip}(\bar{g}^{(t)}, C) + \mathcal{N}(0, \sigma^2 I_d))$ where $\sigma = \sqrt{2 \ln(1.25/\delta)} \cdot C/\epsilon$. (6) Output $\tilde{g}^{(t)}$ if sufficient verified gradients are aggregated, else $\perp$.

Theorem 1 (Semi-Honest Security): For every non-uniform PPT adversary $\mathcal{A}$ controlling any subset $\mathcal{C}_{\text{corrupt}} \subseteq \mathcal{C}$ of malicious clients and at most one semi-honest server, there exists a non-uniform PPT simulator $\mathcal{S}$ such that $\{\text{Ideal}_{\mathcal{F}_{\text{Fortress}}, \mathcal{S}(z)}(\{g_i^{(t)}\}_{i=1}^{|\mathcal{C}|}, \lambda, \kappa)\}_{z, \{g_i^{(t)}\}_{i=1}^{|\mathcal{C}|}, \lambda, \kappa} \approx_c \{\text{Real}_{\Pi, \mathcal{A}(z)}(\{g_i^{(t)}\}_{i=1}^{|\mathcal{C}|}, \lambda, \kappa)\}_{z, \{g_i^{(t)}\}_{i=1}^{|\mathcal{C}|}, \lambda, \kappa}$ except with probability $O(|\mathcal{C}_{\text{corrupt}}| \cdot (2^{-\lambda} + 2^{-\kappa}))$, where Π denotes the Fortress protocol, λ is the computational security parameter, κ is the statistical security parameter, and $\approx_c$ denotes computational indistinguishability.

Proof. Simulator Construction: The simulator $\mathcal{S}$ extracts corrupted clients' inputs by reconstructing $\bar{g}_{i,j}^{(t)} = \bigoplus_{k=0}^{w_i^{(t)}-1} x_{i,j,k}^{(0)} \oplus x_{i,j,k}^{(1)}$ and uses zero vectors for honest clients, compensating via $\mathcal{F}_{\text{Fortress}}$ output while generating synthetic Boolean shares $x_{i,j,k}^{(b)} \xleftarrow{\$} \{0, 1\}$ and valid correlations (q_j, t_j) with $t_j = q_j + x_{i,j,k}^{(1)} \cdot \Delta^{(i)}$ and (α_k, β_k) with $\beta_k = \alpha_k^2 \bmod 2^v$.

Hybrid Arguments: We analyze three hybrids: H_0 (real execution), H_1 (zero inputs for clients with correction), and H_2 (ideal computation). The transition $H_0 \rightarrow H_1$ relies on Boolean shares being uniformly distributed over $\mathbb{F}_2^{w_i^{(t)}}$ and verification challenges $\{\chi_j\}_{j=1}^n$ sampled after correlation submission, giving distinguishing advantage $|\mathcal{C}_{\text{corrupt}}| \cdot 2^{-\lambda}$. The transition $H_1 \rightarrow H_2$ uses correlation verification soundness from Lemmas 3 and 4 with advantage $|\mathcal{C}_{\text{corrupt}}| \cdot (2^{-\lambda} + 2^{-\kappa})$. Since H_2 corresponds to ideal execution, the overall distinguishing advantage is $3 \cdot |\mathcal{C}_{\text{corrupt}}| \cdot (2^{-\lambda} + 2^{-\kappa})$, negligible in security parameters. $\square$

E. Malicious Security Proofs

For malicious privacy, we follow the definition where a protocol Π with p input parties and $n - p$ helpers achieves (t, s) -malicious privacy if for every PPT adversary $\mathcal{A}$ controlling at most t input parties and s helpers, there exists a PPT simulator $\mathcal{S}$ such that the ideal and real world distributions are computationally indistinguishable, where the ideal functionality allows the adversary to specify output corruption after seeing the honest computation result.

Theorem 2 (Malicious Privacy via Transcript Emulation): The Fortress protocol $(|C| + 2, 1)$ -maliciously privately computes the $|C|$ -party functionality $\mathcal{F}_{\text{Fortress}}$ in the $(\mathcal{F}_{\text{CoinFlip}}, \mathcal{F}_{\text{RO}})$ -hybrid model against static malicious adversaries, where at most one of the two servers $\mathcal{S}_0, \mathcal{S}_1$ can be corrupted along with any proper subset of clients $\mathcal{C}_{\text{corrupt}} \subsetneq \mathcal{C}$ (i.e., assuming at least one honest client).

Proof. Simulator Construction: The simulator $\mathcal{S}$ operates by simulating a constrained adversary $\mathcal{A}'$ that follows protocol specifications as determined by client transcript predictions. All operations on client i 's input are deterministic given $(g_i^{(t)}, \rho_i, \chi)$ where $g_i^{(t)}$ represents compressed gradient shares, ρ_i denotes correlations from Protocols 3 and 4, and χ captures verification challenges from Protocol 5. The transcript prediction function $\mathcal{T}_i(g_i^{(t)}, \rho_i, \chi) \rightarrow T_i$ generates expected server messages across all phases: correlation verification via linear combinations $\hat{t} = \sum_{j=1}^n t_j \cdot \chi_j$ and sacrifice-based verification $\tau^2 \beta_k - \hat{\beta}_k - 2\tau e_k \alpha_k + e_k^2$, secure aggregation through conversion $z_j^{(b)} = \sum_{k=0}^{w-1} 2^k (x_{j,k}^{(b)} - 2y_{j,k}^{(b)})$, and DP including polynomial approximation and secure comparison.

Hybrid Arguments: Hash-based verification ensures malicious detection before privacy compromise. Each client computes $H_i = \text{Hash}(T_i \setminus \mathcal{R}_i)$ while the honest server maintains $H_s = \text{Hash}(\mathcal{M} \setminus \mathcal{R})$. Clients submit correlations without knowing challenges χ , maintaining verification soundness from Lemmas 3 and 4. After challenge revelation and transcript completion, any digest mismatch $H_i \neq H_s$ triggers abort before output revelation, preserving privacy with probability $1 - 2^{-\lambda}$ due to collision-resistant hashing. Security follows through hybrid arguments: H_0 (real world) $\rightarrow H_1$ (replace $\mathcal{A}$ with transcript-following $\mathcal{A}'$) $\rightarrow H_2$ (zero inputs with functionality correction), establishing malicious privacy where client privacy is preserved even under server manipulation. $\square$

F. Differential Privacy Guarantees

Theorem 3 (DP of Fortress Protocol): The Fortress protocol satisfies (ϵ, δ) -DP where noise parameter $\sigma = \sqrt{2 \ln(1.25/\delta)} \cdot C/\epsilon$, clipping bound C , and privacy parameters (ϵ, δ) are public. For T training rounds, the protocol achieves (ϵ', δ') -DP under advanced composition with $\epsilon' = \sqrt{2T \ln(1/\delta')} \cdot \epsilon + T\epsilon(\epsilon + 1)$ for appropriately chosen δ' .

Proof: After secure gradient clipping in Protocol 6, all aggregated gradients satisfy $\|\bar{g}_{\mathcal{D}}^{(t)}\|_2 \leq C$. For neighboring datasets $\mathcal{D}$ and $\mathcal{D}'$ differing in one client, the ℓ_2 -sensitivity is $\Delta_2 = \|\bar{g}_{\mathcal{D}}^{(t)} - \bar{g}_{\mathcal{D}'}^{(t)}\|_2 \leq 2C$ since each client's contribution is clipped to norm at most C . The Gaussian noise $n^{(t)} \sim$

$\mathcal{N}(0, \sigma^2 I_d)$ with $\sigma = \sqrt{2 \ln(1.25/\delta)} \cdot C/\epsilon$ provides (ϵ, δ) -DP by the standard mechanism. Server-side noise generation maintains the required distribution: each server samples $n^{(t),(b)} \sim \mathcal{N}(0, (\sigma/\sqrt{2})^2 I_d)$ so that combined noise $\langle n^{(t)} \rangle^{(b)} = n^{(t),(0)} + n^{(t),(1)} \sim \mathcal{N}(0, \sigma^2 I_d)$ has correct calibration. Secure computation preserves DP because operations on secret-shared values maintain the same sensitivity bound $\Delta_2 = 2C$ and reveal only the final noised result without additional leakage. Multi-round composition over T rounds follows the advanced composition theorem with $\epsilon' = \sqrt{2T \ln(1/\delta')} \cdot \epsilon + T\epsilon(\epsilon + 1)$, significantly improving over basic composition's $T\epsilon$ bound. A malicious server attempting to manipulate the model by injecting large magnitude noise or specific adversarial vectors will disrupt the expected statistical properties of the aggregate noise vector. The verification check $\|n^{(t)}\|_2^2 - \mathbb{E}[\|n^{(t)}\|_2^2]\| \leq \tau$ ensures that the injected noise follows the expected Chi-squared distribution. Since the noise is secret-shared before verification, a malicious server cannot adaptively modify its shares to satisfy this check while deviating from the Gaussian distribution, detecting arbitrary noise injection with high probability. $\square$

V. EVALUATION

A. Experimental Setup

Implementation and Environment: We implement Fortress using PyTorch 2.1.1 in Python 3.8 with Opacus 1.1.2 for DP [36]. Cryptographic operations use the cryptography library [37] for hash functions and galois [38] for finite field arithmetic. Experiments run on dual servers (Ubuntu 18.04, Xeon Gold 6133@2.5 GHz, 128 GB RAM, 6×RTX 4090) coordinating secure computation between $\mathcal{S}_0$ and $\mathcal{S}_1$. Client processes simulate distributed settings with 20 ms latency.

Datasets and Models: We evaluate Fortress across three domains. For image classification, we use CIFAR-10 [39] (60,000 32×32 images, 10 classes) distributed among 100 clients and Tiny-ImageNet [40] (100,000 64×64 images, 200 classes) among 50 clients, tested with VGG-16 [41] and ResNet-34 [42]. For natural language processing, we employ SST-2 [43] (67,349 movie reviews) for binary sentiment classification using transformer architectures [44]. For speech recognition, we utilize Speech Commands [45] (105,829 audio utterances, 35 words) with ResNet-18 [42]. All datasets use uniform distribution with balanced classes across clients.

Baselines: We compare against five secure distributed learning systems: SecAgg [19], PrivateFL [15], RoFL [26], ELSA [24], and Prio+ [23]. These represent the spectrum from basic aggregation to comprehensive multi-level protection, enabling evaluation against both single-mechanism and hybrid approaches.

Parameter Configuration: Our parameter choices combine theoretical considerations with empirical validation. Cryptographic parameters follow standard practice: computational security $\lambda = 128$ bits, statistical security $\kappa = 40$ bits. Adaptive compression uses exponential smoothing coefficients $\alpha = 0.7$, $\beta = 0.3$. We enforce a strict global bit-width limit $W_{\max} = 8$ bits. The effective bit-width $w_i^{(t)}$ (including sign bit) varies

TABLE II
PER-CLIENT COMMUNICATION OVERHEAD ACROSS DIFFERENT DATASETS AND MODELS (MB)

Method	CIFAR-10 (VGG-16)	Tiny-ImageNet (ResNet-34)	SST-2 (LSTM)	Speech Commands (ResNet-18)
SecAgg	142.8	168.2	125.4	98.7
RoFL	216.7	255.3	190.3	149.8
PrivateFL	121.8	143.5	107.0	84.2
Prio+	162.3	191.2	142.6	112.3
ELSA	46.8	52.1	37.2	29.4
Fortress	7.2	8.4	5.9	4.7

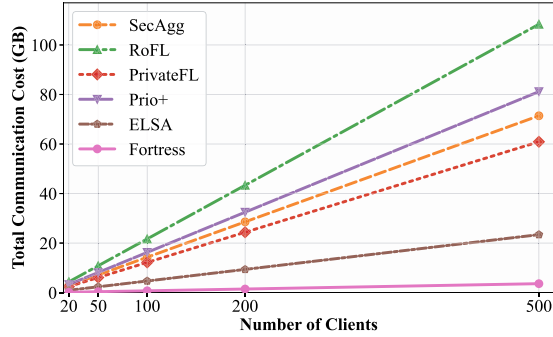


Fig. 2. Total Communication Cost with Varying Number of Clients on CIFAR-10 (VGG-16) (GB).

dynamically within $[2, W_{\max}]$ based on local gradient updates. Privacy budget $\epsilon = 2.0$, $\delta = 10^{-5}$ follows standard federated DP practice, balancing privacy strength against model utility. Clipping threshold is $C = 1.0$. The degree-4 Chebyshev polynomial achieves approximation error below 2^{-40} ; higher degrees provide only marginal accuracy gains at the cost of additional secure multiplication rounds. Training hyperparameters: learning rate $\eta = 0.01$, local batch size $B = 32$, local epochs $E = 5$, total rounds $T = 100$.

B. Communication Efficiency Evaluation

Per-Client Communication Overhead Across Datasets and Models: Table II shows that Fortress achieves the lowest communication overhead among all methods, demonstrating $6.2\times$ – $32.3\times$ reduction compared to baseline methods. For CIFAR-10 with VGG-16, Fortress requires only 7.2MB compared to ELSA’s 46.8MB and RoFL’s 216.7MB, achieving $6.5\times$ and $30.1\times$ improvements respectively over these state-of-the-art secure methods. Similar efficiency gains are observed across all model architectures: SST-2 with LSTM shows $6.3\times$ improvement over ELSA (5.9MB vs 37.2MB), while Speech Commands with ResNet-18 achieves $6.3\times$ improvement (4.7 MB vs 29.4 MB). The efficiency gain stems from adaptive bit-width compression (4–6 bits based on gradient importance) combined with Boolean secret sharing, outperforming uniform compression approaches.

Communication Scalability with Varying Client Numbers: Fig. 2 demonstrates superior scalability as client numbers increase from 20 to 500. Fortress scales from 0.14 GB to

TABLE III
WALL-CLOCK TIME PER ROUND (S) AND CONVERGENCE ACCURACY (%) ON CIFAR-10 (VGG-16) UNDER UPLINK BANDWIDTH CONSTRAINTS

Method	1 Mbps		5 Mbps		10 Mbps		50 Mbps		100 Mbps	
	Time	Acc.	Time	Acc.	Time	Acc.	Time	Acc.	Time	Acc.
SecAgg	1155.2	90.9	231.4	91.0	115.8	91.0	23.4	91.0	21.3	91.0
RoFL	1753.6	90.7	351.2	90.8	175.8	90.8	35.6	90.8	30.1	90.8
PrivateFL	985.4	89.2	197.3	89.3	98.9	89.4	20.2	89.4	19.8	89.4
Prio+	1313.8	90.5	263.1	90.5	131.7	90.6	27.0	90.6	25.4	90.6
ELSA	378.9	90.1	76.0	90.1	38.2	90.2	22.1	90.2	21.1	90.2
Fortress	58.3	87.8	23.4	87.8	20.5	87.8	18.9	87.8	18.5	87.8

TABLE IV
OVERALL EXECUTION TIME PER TRAINING ROUND ACROSS DIFFERENT DATASETS AND MODELS (SECONDS)

Method	CIFAR-10 (VGG-16)	Tiny-ImageNet (ResNet-34)	SST-2 (LSTM)	Speech Commands (ResNet-18)
SecAgg	35.7	46.4	29.1	21.2
RoFL	52.8	68.7	43.2	31.5
PrivateFL	25.8	33.6	21.1	15.4
Prio+	28.6	37.2	23.4	17.1
ELSA	21.4	27.8	17.5	12.8
Fortress	18.5	24.1	15.1	11.0

All experiments conducted with 100 clients, $\epsilon = 2.0$, $C = 1.0$.

3.6 GB, maintaining the gentlest slope (7.2 MB/client) compared to recent methods: ELSA (46.8 MB/client) and Prio+ (162.3 MB/client). At 500 clients, Fortress achieves $6.5\times$ improvement over ELSA (3.6GB vs 23.4GB) and $22.5\times$ over Prio+ (3.6 GB vs 81.15 GB), with efficient correlation batch verification amortizing costs across multiple clients.

Performance Under Varying Bandwidth Constraints: Table III reports wall-clock time per training round and convergence accuracy on CIFAR-10 (VGG-16) with 100 clients under uplink bandwidth $B \in \{1, 5, 10, 50, 100\}$ Mbps and fixed round-trip latency 20 ms. Fortress achieves the shortest per-round time across all settings. At 1 Mbps, Fortress completes each round in 58.3 s, compared to 378.9 s for ELSA ($6.5\times$ slower) and 1753.6 s for RoFL ($30.1\times$ slower), because its compression directly reduces the time dominated by uplink transmission. Convergence accuracy is unaffected by bandwidth since compression is performed before transmission.

C. Computational Efficiency Evaluation

Table IV presents the total execution time per training round with 100 clients across various datasets and models. Fortress requires 18.5 seconds per round on CIFAR-10, achieving the best performance among all secure methods. Compared to existing secure approaches, Fortress demonstrates competitive efficiency: ELSA requires 21.4 seconds ($1.2\times$ slower), PrivateFL needs 25.8 seconds ($1.4\times$ slower), Prio+ takes 28.6 seconds ($1.5\times$ slower), while SecAgg exhibits 35.7 seconds ($1.9\times$ slower). RoFL shows the highest overhead at 52.8 seconds ($2.9\times$ slower than Fortress). Fortress achieves comprehensive multi-level security with the lowest computational overhead across all model architectures. The superior performance stems from efficient batch verification mechanisms and client-side correlation generation that minimize expensive server-to-server coordination. While existing specialized solutions like ELSA

TABLE V
EXECUTION TIME PER TRAINING ROUND (SECONDS) WITH NUMBER OF
CLIENTS ON CIFAR-10 (VGG-16)

Method	$n = 20$	$n = 50$	$n = 100$	$n = 200$	$n = 500$
SecAgg	12.3	22.1	35.7	68.4	186.3
RoFL	18.6	34.2	52.8	103.7	284.1
PrivateFL	9.4	16.8	25.8	49.3	131.7
Prio+	10.2	18.7	28.6	55.8	146.2
ELSA	8.1	14.3	21.4	41.6	109.8
Fortress	6.7	11.9	18.5	35.2	91.4

TABLE VI
GRADIENT INVERSION ATTACK RESISTANCE ACROSS DATASETS AND MODELS

Method	CIFAR-10		Tiny-ImageNet		SST-2		Speech Commands	
	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
SecAgg	15.2	0.31	14.8	0.29	13.6	0.26	12.9	0.24
RoFL	12.8	0.22	12.3	0.21	11.7	0.19	11.2	0.18
PrivateFL	12.1	0.20	11.6	0.19	10.9	0.17	10.4	0.16
Prio+	13.5	0.25	13.1	0.23	12.4	0.21	11.8	0.20
ELSA	11.8	0.19	11.4	0.18	10.7	0.16	10.2	0.15
Fortress	9.7	0.14	9.3	0.13	8.8	0.12	8.4	0.11

Lower values indicate better protection.

provide partial security with reasonable efficiency, Fortress delivers integrated multi-level protection with optimal computational costs, making it the most practical choice.

To verify that this advantage holds across client counts, Table V reports execution time per round for $n \in \{20, 50, 100, 200, 500\}$ on CIFAR-10 (VGG-16). Fortress maintains the lowest execution time across all client counts. At $n = 500$, Fortress requires 91.4 seconds per round, outperforming ELSA (109.8 s, $1.2\times$ slower), Prio+ (146.2 s, $1.6\times$ slower), and RoFL (284.1 s, $3.1\times$ slower). All methods scale approximately linearly in n , as the dominant cost is server-side aggregation of $O(n \cdot m \cdot w)$. Fortress achieves a consistently shallower slope because adaptive compression reduces w and client-side correlation generation eliminates expensive server-to-server interaction.

D. Privacy Protection Capabilities

Gradient Inversion Attack Resistance: Table VI presents resistance against gradient inversion attacks using Deep Gradient Leakage (DGL) [7] and Inverting Gradients (IG) [46] attacks. We measure reconstruction quality using Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM), with lower values indicating better protection. Fortress achieves the strongest defense across all datasets and models. On CIFAR-10, existing secure methods allow moderate reconstruction quality, while Fortress reduces reconstruction quality to PSNR 9.7 dB and SSIM 0.14, effectively preventing successful attacks. The superior protection stems from multi-level defense: secure aggregation, DP noise, and adaptive compression. Similar trends hold across Tiny-ImageNet, SST-2, and Speech Commands, with Fortress consistently achieving the lowest reconstruction metrics compared to all baselines.

Membership Inference Attack Defense: Table VII evaluates defense against membership inference attacks using shadow

TABLE VII
MEMBERSHIP INFERENCE ATTACK DEFENSE ACROSS DATASETS AND MODELS

Method	CIFAR-10		Tiny-ImageNet		SST-2		Speech Commands	
	Acc.	AUC	Acc.	AUC	Acc.	AUC	Acc.	AUC
SecAgg	68.7%	0.72	67.1%	0.71	70.4%	0.75	66.2%	0.70
RoFL	65.4%	0.69	64.2%	0.68	67.1%	0.71	63.8%	0.67
PrivateFL	57.3%	0.59	56.8%	0.58	59.1%	0.61	55.9%	0.57
Prio+	61.2%	0.64	60.5%	0.63	62.8%	0.66	59.3%	0.62
ELSA	59.8%	0.61	58.9%	0.60	61.4%	0.63	57.6%	0.59
Fortress	52.8%	0.54	52.1%	0.53	53.6%	0.55	51.9%	0.52

Random guessing baseline: 50% accuracy, 0.5 AUC.

TABLE VIII
MODEL ACCURACY UNDER RANDOM NOISE ATTACKS (20% MALICIOUS
CLIENTS)

Method	CIFAR-10		Tiny-ImageNet		SST-2		Speech Commands	
	Clean	Attack	Clean	Attack	Clean	Attack	Clean	Attack
SecAgg	91.0%	41.8%	67.7%	27.9%	87.4%	38.6%	95.2%	41.2%
RoFL	90.8%	86.1%	67.5%	62.3%	87.2%	83.4%	95.0%	91.8%
PrivateFL	89.4%	78.2%	65.8%	54.1%	86.1%	76.9%	94.2%	82.6%
Prio+	90.6%	82.4%	67.1%	58.7%	86.9%	79.3%	94.8%	87.2%
ELSA	90.2%	84.3%	66.8%	60.2%	86.7%	81.1%	94.6%	89.1%
Fortress	90.1%	87.8%	66.5%	63.7%	86.8%	84.2%	94.4%	92.4%

Clean: performance without attacks; Attack: performance with 20% malicious clients.

model methodology [10]. We measure attack accuracy and Area Under the Curve (AUC), with values closer to random guessing (50% accuracy, 0.5 AUC) indicating better protection. Fortress provides the strongest defense, consistently approaching random guessing performance across all datasets. On CIFAR-10, baseline secure methods suffer significant privacy leakage, while Fortress achieves near-optimal protection (52.8% accuracy, 0.54 AUC). Methods without DP (SecAgg, RoFL) show substantial vulnerability, while PrivateFL demonstrates moderate protection through DP. Fortress's comprehensive approach combining cryptographic protection with integrated DP achieves the best results across all evaluation scenarios, effectively preventing membership inference while maintaining model utility.

E. Malicious Resilience Evaluation

We evaluate the malicious resilience of Fortress against random noise attacks, where malicious clients inject random gradients to disrupt training. This evaluation compares all baseline methods across different datasets and models, followed by scalability analysis under varying threat levels.

Defense Against Random Noise Attacks: Table VIII presents model accuracy under random noise attacks where 20% of clients send random gradients instead of legitimate updates. This represents a form of malicious behavior testing the core robustness mechanisms of each approach. Methods without malicious defense mechanisms suffer catastrophic accuracy degradation. SecAgg experiences severe drops across all datasets, with CIFAR-10 accuracy falling from 91.0% to approximately 42%. Robust methods show substantially better resilience: RoFL maintains 86.1% accuracy through statistical filtering, while Prio+ and ELSA achieve 82.4% and 84.3% respectively. Fortress provides strong robustness across all datasets, achieving 87.8%

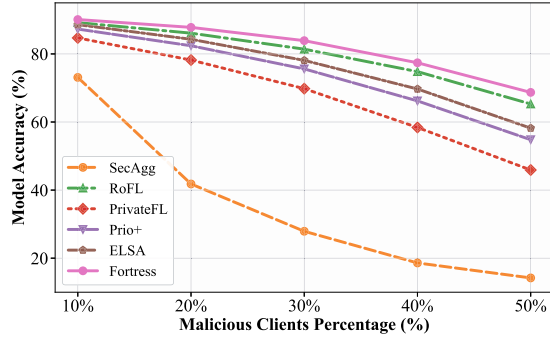


Fig. 3. Malicious Defense Scalability Under Random Noise Attacks on CIFAR-10 (VGG-16).

TABLE IX
MODEL ACCURACY UNDER STRONGER BYZANTINE ATTACKS

Method	Clean		Label Flipping		Min-Max		Min-Sum	
	Acc.	-	Acc.	Drop	Acc.	Drop	Acc.	Drop
SecAgg	91.0	-	38.2	-52.8	31.7	-59.3	34.1	-56.9
RoFL	90.8	-	81.3	-9.5	74.2	-16.6	76.8	-14.0
PrivateFL	89.4	-	74.6	-14.8	66.3	-23.1	69.1	-20.3
Prio+	90.6	-	79.1	-11.5	71.8	-18.8	73.9	-16.7
ELSA	90.2	-	80.4	-9.8	73.6	-16.6	75.7	-14.5
Fortress	90.1	-	84.7	-5.4	79.3	-10.8	81.6	-8.5

accuracy on CIFAR-10 (only 2.3 point drop from its clean performance of 90.1%) and similar minimal degradation on other datasets. This superior performance stems from correlation verification and transcript-based verification that effectively filter malicious contributions before aggregation. The marginally lower clean accuracy of Fortress relative to non-DP baselines (e.g., 90.1% vs. 91.0% for SecAgg on CIFAR-10) is attributable to the Gaussian noise added. This cost is the utility–privacy tradeoff: Fortress reduces membership inference accuracy to 52.8%, whereas baselines that omit DP remain substantially more vulnerable.

Scalability of Malicious Defense Under Varying Threat Levels: Fig. 3 evaluates defense effectiveness with increasing numbers of malicious clients using CIFAR-10 dataset with VGG-16 model, demonstrating the practical limits of different defense mechanisms. Methods without defense exhibit rapid degradation as adversarial participation increases. Robust methods demonstrate varying scalability: RoFL maintains 81.4% accuracy at 30% malicious clients but degrades to 65.3% at 50%, while PrivateFL shows rapid degradation to 69.8% and 45.9% respectively. Fortress exhibits superior scalability across all threat levels, maintaining 83.9% accuracy with 30% malicious clients and 68.7% even under extreme conditions with 50% malicious participation. The superior scalability stems from multi-layered defense mechanisms that remain effective as adversarial participation increases.

Defense Against Stronger Byzantine Attacks: Table IX extends the robustness evaluation to three stronger Byzantine attack strategies: 1) Label Flipping [12], [47]: Malicious clients train on locally relabeled data (e.g., class $i \rightarrow$ class $9 - i$ for CIFAR-10) and submit the resulting poisoned gradients; 2) Min-Max [11], [47]: The adversary crafts a malicious gradient that maximizes

TABLE X
MODEL ACCURACY UNDER NON-IID DATA WITH 20% RANDOM NOISE ATTACKERS ON CIFAR-10 (VGG-16)

Method	$\alpha_{\text{Dir}} = 0.1$		$\alpha_{\text{Dir}} = 0.5$		$\alpha_{\text{Dir}} = 1.0$	
	Clean	Attack	Clean	Attack	Clean	Attack
SecAgg	78.3	33.1	85.6	38.7	88.9	40.2
RoFL	77.9	72.4	85.1	81.6	88.6	85.2
PrivateFL	76.4	67.2	83.8	74.9	87.2	77.3
Prio+	77.6	70.1	84.9	79.3	88.3	82.1
ELSA	77.2	71.8	84.7	80.9	88.1	83.7
Fortress	77.1	73.6	84.5	82.1	87.8	86.9

TABLE XI
MODEL ACCURACY ON CIFAR-10 (VGG-16) UNDER PRIVACY BUDGETS

ϵ	Accuracy (%)	Drop vs. No-DP
0.5	78.3	-11.8
1.0	83.6	-6.5
2.0	87.8	-2.3
4.0	89.2	-0.9
8.0	89.9	-0.2
No DP	90.1	-

TABLE XII
MODEL ACCURACY AND PER-CLIENT COMMUNICATION ON CIFAR-10 (VGG-16) UNDER VARYING MAXIMUM BIT-WIDTH

W_{max} (bits)	Accuracy (%)	Comm. (MB/client)
2	81.4	1.8
4	85.9	3.2
6	87.4	5.0
8	87.8	7.2
32 (no compression)	88.1	28.8

model degradation while minimizing its distance from the average of benign gradients; 3) Min-Sum [11], [47]: Similar to Min-Max but minimizes the sum of pairwise distances to benign gradients. Fortress achieves the smallest accuracy drop across all three attack types.

Robustness Under Non-IID Data: Table X evaluates robustness under heterogeneous data distributions using Dirichlet partitioning [48] with concentration parameter $\alpha_{\text{Dir}} \in \{0.1, 0.5, 1.0\}$ (lower values indicate higher heterogeneity), combined with 20% random noise attackers. Fortress maintains the highest attack-accuracy across all heterogeneity levels. The correlation and transcript verification mechanisms are agnostic to the data distribution: they operate on gradient shares rather than raw gradients, so their filtering effectiveness does not degrade as data becomes more heterogeneous.

F. Utility–Privacy Trade-Off Analysis

Impact of Privacy Budget (ϵ, δ) on Model Accuracy: Table XI evaluates model accuracy on CIFAR-10 (VGG-16) under varying privacy budgets $\epsilon \in \{0.5, 1.0, 2.0, 4.0, 8.0\}$ with fixed $\delta = 10^{-5}$. At $\epsilon = 2.0$, the accuracy drop is only 2.3 percentage points relative to the non-private baseline, confirming that privacy is achievable with modest utility cost.

Impact of Compression Level on Model Accuracy: Table XII evaluates accuracy under varying maximum bit-width $W_{\text{max}} \in$

TABLE XIII
MODEL ACCURACY (%) ON CIFAR-10 (VGG-16) UNDER JOINT COMPRESSION
AND PRIVACY BUDGET SETTINGS

$W_{\max}$ (bits)	Privacy Budget ϵ		
	1.0	2.0	4.0
4	80.1	85.9	87.8
6	82.6	87.4	88.9
8	83.6	87.8	89.2

TABLE XIV
ABLATION STUDY ON EXPONENTIAL SMOOTHING COEFFICIENTS (α, β) ON
CIFAR-10 (VGG-16)

α	β	Comm. (MB)	Accuracy (%)	Time (s/round)
0.3	0.7	10.3	87.2	20.1
0.5	0.5	9.8	87.5	19.6
0.7	0.3	7.2	87.8	18.5
0.9	0.1	6.6	86.9	17.8

$\{2, 4, 6, 8\}$ bits with fixed $\epsilon = 2.0$. Even at $W_{\max} = 4$ bits, Fortress achieves 85.9% accuracy with only 3.2MB per-client communication, a $9\times$ reduction over the uncompressed baseline. The gap relative to no compression narrows to 0.3 percentage points at $W_{\max} = 8$, confirming that adaptive bit-width compression effectively preserves gradient fidelity.

Joint Effect of Compression and Privacy Budget: Table XIII reports a grid search over $\epsilon \in \{1.0, 2.0, 4.0\}$ and $W_{\max} \in \{4, 6, 8\}$. Two observations emerge. First, the accuracy losses from compression and DP noise are approximately additive. Second, at ($\epsilon = 2.0$, $W_{\max} = 8$), Fortress achieves 87.8% accuracy within 2.3 points of the non-private uncompressed baseline, while reducing communication by $6.5\times$ over ELSA.

G. Ablation Study on Compression Hyperparameters

We evaluate the sensitivity of Fortress to the exponential smoothing coefficients ($\alpha, \beta = 1 - \alpha$) in the adaptive compression level update. Four settings $\alpha \in \{0.3, 0.5, 0.7, 0.9\}$ are evaluated on CIFAR-10 (VGG-16) with $n = 100$ clients, $\epsilon = 2.0$, $\delta = 10^{-5}$, $W_{\max} = 8$, and $T = 100$ rounds. Table XIV reports per-client communication overhead, convergence accuracy, and execution time per round. The results yield two observations. First, accuracy varies by only 0.9 percentage points across $\alpha \in [0.3, 0.9]$, confirming that the mechanism is insensitive to the specific coefficient values. Second, $\alpha = 0.7$ achieves the best accuracy because a moderately high α allows the compression level to adapt gradually, avoiding both over-compression that loses gradient directional information and under-compression that wastes bandwidth.

VI. RELATED WORK

Single-Level Security Frameworks for Distributed Learning: Cryptographic aggregation protocols [13], [14], [19] provide gradient privacy through secret sharing, achieving efficient communication but offering no protection against inference attacks or malicious clients. Recent work further demonstrates the

potency of gradient-level threats: Loki [7] shows that gradient inversion attacks remain effective at scale even under FedAvg, and Dager [46] extends exact gradient inversion to larger models. DP approaches [15], [16], [49] prevent membership inference through noise addition with formal guarantees, yet operate on plaintext gradients and suffer utility degradation from client-side noise composition. A systematic study [9] further demonstrates that membership inference risk persists in distributed settings. Malicious resilience algorithms [17], [50] filter malicious updates via statistical analysis but require plaintext access incompatible with encryption. Communication optimization techniques [51], [52], [53] achieve significant compression in plaintext settings but lose effectiveness when combined with cryptography due to padding requirements or information leakage through compression metadata. These specialized solutions excel within their domains but operate under conflicting assumptions: cryptographic methods need fixed-precision finite field arithmetic, DP requires gradient norm access, robust aggregation depends on plaintext statistics, and compression exploits structure that encryption obscures.

Multi-Level Security Frameworks for Distributed Learning: Recent research explores combining multiple security mechanisms to achieve comprehensive protection in distributed learning systems. RoFL [26] integrates secure aggregation with malicious robustness by performing statistical filtering operations on homomorphically encrypted gradients, enabling both privacy and robustness at the cost of expensive ciphertext computations. Aditya et al. [54] utilizes trusted execution environments to simultaneously provide confidentiality, integrity, and robustness guarantees within hardware-protected enclaves. Prio+ [23] combines verifiable aggregation with DP for statistical queries, using zero-knowledge proofs to ensure correctness while adding noise for privacy protection. ELSA [24] merges gradient compression with secure aggregation through specialized secure multiparty computation protocols that operate on quantized values within secret-shared domains. Aion [55] proposes a secure aggregation scheme tolerating malicious participants but does not integrate DP or gradient reconstruction defense. [56] confirms that cryptographic protection combined with DP constitutes the most robust defense class, precisely the combination that Fortress realizes. While multi-level approaches advance comprehensive security, they suffer from multiplicative overhead, missed cross-layer optimization opportunities, and prohibitive costs for large-scale deployment.

VII. CONCLUSION

Fortress transforms secure distributed learning through unified rather than independent security mechanisms. By integrating adaptive gradient compression with Boolean secret sharing, server-side DP through polynomial approximation, and lightweight transcript verification, we achieve significant efficiency gains while maintaining comprehensive protection. This integrated approach reduces communication overhead by $6.2\times$ – $32.3\times$ and eliminates expensive cryptographic primitives, making secure distributed learning practical at scale.

REFERENCES

- [1] J. Weng, J. Weng, J. Zhang, M. Li, Y. Zhang, and W. Luo, "Deepchain: Auditable and privacy-preserving deep learning with blockchain-based incentive," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2438–2455, Sep./Oct. 2021.
- [2] X. Zhang, L. Fan, S. Wang, W. Li, K. Chen, and Q. Yang, "A game-theoretic framework for privacy-preserving federated learning," *ACM Trans. Intell. Syst. Technol.*, vol. 15, pp. 1–35, 2024.
- [3] A. Tuladhar, D. Rajashekar, and N. D. Forkert, "Distributed learning in healthcare," *Trends Artif. Intell. Big Data E-Health*, vol. 9, pp. 183–212, 2023.
- [4] Z. Liu, X. Zhang, W. Liu, W. Chen, Y. Li, and Y. Zhou, "Application and optimization of digital situated teaching in university finance courses from a constructivist perspective: An analysis based on machine learning algorithms," *Educ. Inf. Technol.*, vol. 30, pp. 18059–18088, 2025.
- [5] C. P. Filho et al., "A systematic literature review on distributed machine learning in edge computing," *Sensors*, vol. 22, 2022, Art. no. 2665.
- [6] Official journal of the European union. general data protection regulation, 2016. Accessed: Feb. 12, 2021. [Online]. Available: <https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0679>
- [7] J. C. Zhao, A. Sharma, A. R. Elkordy, Y. H. Ezzeldin, S. Avestimehr, and S. Bagchi, "Loki: Large-scale data reconstruction attack against federated learning through model manipulation," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 1287–1305.
- [8] Q. Tan, Q. Li, Y. Zhao, Z. Liu, X. Guo, and K. Xu, "Defending against data reconstruction attacks in federated learning: An information theory approach," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 325–342.
- [9] H. Chang, B. Edwards, A. S. Paul, and R. Shokri, "Efficient privacy auditing in federated learning," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 307–323.
- [10] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 739–753.
- [11] V. Shejwalkar and A. Houmansadr, "Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2021.
- [12] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to {Byzantine-Robust} federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1605–1622.
- [13] Y. Ma, J. Woods, S. Angel, A. Polychroniadou, and T. Rabin, "Flamingo: Multi-round single-server secure aggregation with applications to private federated learning," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 477–496.
- [14] S. Sav et al., "POSEIDON: Privacy-preserving federated neural network learning," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp., NDSS*, 2021.
- [15] Y. Yang, B. Hui, H. Yuan, N. Gong, and Y. Cao, "Proc. {PrivateFL}: Accurate, differentially private federated learning via personalized data transformation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 1595–1612.
- [16] A. Girgis, D. Data, S. Diggavi, P. Kairouz, and A. T. Suresh, "Shuffled model of differential privacy in federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2021, pp. 2521–2529.
- [17] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp., NDSS*, 2021.
- [18] C. Xie, M. Chen, P.-Y. Chen, and B. Li, "CRFL: Certifiably robust federated learning against backdoor attacks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 11372–11382.
- [19] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.
- [20] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly) logarithmic overhead," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 1253–1269.
- [21] J. So, B. Güler, and A. S. Avestimehr, "Turbo-aggregate: Breaking the quadratic aggregation barrier in secure federated learning," *IEEE J. Sel. Areas Inf. Theory*, vol. 2, no. 1, pp. 479–489, Mar. 2021.
- [22] J. Dong, A. Roth, and W. J. Su, "Gaussian differential privacy," *J. Roy. Stat. Soc. Ser. B, Stat. Methodol.*, vol. 84, pp. 3–37, 2022.
- [23] S. Addanki, K. Garbe, E. Jaffe, R. Ostrovsky, and A. Polychroniadou, "Priort+: Privacy preserving aggregate statistics via boolean shares," in *Proc. Int. Conf. Secur. Cryptogr. Netw.*, 2022, pp. 516–539.
- [24] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "Elsa: Secure aggregation for federated learning with malicious actors," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 1961–1979.
- [25] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [26] L. Burkharter, H. Lycklama, A. Viand, N. Küchler, and A. Hithnawi, "RoFL: Attestable robustness for secure federated learning," 2021, *arXiv:2107.03311*.
- [27] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [28] G. Asharov, Y. Lindell, T. Schneider, and M. Zohner, "More efficient oblivious transfer and extensions for faster secure computation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 535–548.
- [29] V. Kolesnikov and R. Kumaresan, "Improved ot extension for transferring short secrets," in *Proc. Annu. Cryptology Conf.*, 2013, pp. 54–70.
- [30] D. Rathee et al., "CrypTFlow2: Practical 2-party secure inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 325–342.
- [31] B. Mennink, "Secure distributed modular exponentiation: Systematic analysis and new results," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4188–4197, 2023.
- [32] J. G. Dhokratat et al., "A framework for privacy-preserving multiparty computation with homomorphic encryption and zero-knowledge proofs," *Informatica*, vol. 48, pp. 1–13, 2024.
- [33] P. Arora, A. Tapikar, A. Aryan, V. A. Manish, and V. Sarasvathi, "Fed-shield: Privacy preservation for blockchain enabled federated learning with homomorphic encryption and zero-knowledge proof," in *Proc. Int. Conf. Mach. Learn. Comput.*, 2025, pp. 584–594.
- [34] S. Steffen, B. Bichsel, R. Baumgartner, and M. Vechev, "ZeeStar: Private smart contracts by homomorphic encryption and zero-knowledge proofs," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 179–197.
- [35] M. Mudde, "Chebyshev approximation," Ph.D. dissertation, Faculty Sci. Eng., 2017.
- [36] A. Yousefpouret al., "Opacus: User-friendly differential privacy library in PyTorch," 2021, *arXiv:2109.12298*.
- [37] M. Backes, B. Pfitzmann, and M. Waidner, "A universally composable cryptographic library," *Cryptol. ePrint Arch.*, 2003.
- [38] N. S. Yanofsky, "Galois theory of algorithms," in *Rohit Parikh on Logic, Language and Society*. Berlin, Germany: Springer, 2017, pp. 323–347.
- [39] A. Krizhevskiy et al., "Learning multiple layers of features from tiny images," Toronto, ON, Canada: Univ. of Toronto, Tech. Rep. UTML TR 2009-003, 2009.
- [40] Y. Le et al., "Tiny imagenet visual recognition challenge," *CS 231N*, vol. 7, no. 7, p. 3, 2015.
- [41] H. Qassim, A. Verma, and D. Feinzimer, "Compressed residual-VGG16 CNN model for Big Data places image recognition," in *Proc. IEEE 8th Annu. Comput. Commun. Workshop Conf.*, 2018, pp. 169–175.
- [42] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [43] R. Socher et al., "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2013, pp. 1631–1642.
- [44] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.
- [45] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018, *arXiv:1804.03209*.
- [46] I. Petrov, D. I. Dimitrov, M. Baader, M. N. Müller, and M. Vechev, "Dager: Exact gradient inversion for large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 87801–87830.
- [47] M. Fang, S. Nabavirazavi, Z. Liu, W. Sun, S. S. Iyengar, and H. Yang, "Do we really need to design new Byzantine-robust aggregation rules?," in *Proc. 32nd Annu. Netw. Distrib. Syst. Secur. Symp.*, 2025.
- [48] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," in *Proc. 8th Int. Conf. Learn. Representations, ICLR*, 2020.
- [49] M. Naseri, J. Hayes, and E. D. Cristofaro, "Local and central differential privacy for robustness and privacy in federated learning," in *Proc. 29th Annu. Netw. Distrib. System Secur. Symp.*, 2022.
- [50] P. Rieger, T. D. Nguyen, M. Miettinen, and A. Sadeghi, "DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection," in *Proc. 29th Annu. Netw. Distrib. System Secur. Symp.*, 2022.
- [51] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proc. 15th {USENIX} Symp. Operating Syst. Des. Implementation*, 2021, pp. 19–35.

- [52] Y. Yang, S. Dang, and Z. Zhang, "An adaptive compression and communication framework for wireless federated learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 10835–10854, Dec. 2024.
- [53] Z. Zuo, N. Su, B. Li, and T. Zhang, "Pack: Towards communication-efficient homomorphic encryption in federated learning," in *Proc. ACM Symp. Cloud Comput.*, 2024, pp. 470–486.
- [54] A. P. Kalapaaking, I. Khalil, M. S. Rahman, M. Atiquzzaman, X. Yi, and M. Almashor, "Blockchain-based federated learning with secure aggregation in trusted execution environment for internet-of-things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1703–1714, Feb. 2023.
- [55] Y. Liu et al., "Aion: Robust and efficient {Multi-Round} {Single-Mask} secure aggregation against malicious participants," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 3025–3044.
- [56] V. Carletti, P. Foggia, C. Mazzocca, G. Parrella, and M. Vento, "{SoK}: Gradient inversion attacks in federated learning," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 6439–6459.



Hao Bai (Graduate Student Member, IEEE) received the BE degree in information security from Wuhan University, Wuhan, China, in 2024, where he is currently working toward the PhD degree with the School of Cyber Science and Engineering. His research interests focus on distributed machine learning.

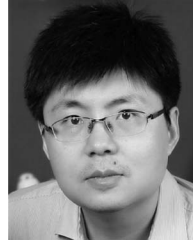


network security, mobile computing, and cloud computing.



learning, and edge computing.

Yuqing Li (Member, IEEE) received the PhD degree in electronic engineering from Shanghai Jiao Tong University, Shanghai, China, in 2019. She is currently an associate professor with the School of Cyber Science and Engineering, Wuhan University, Wuhan, China. From 2019 to 2020, she was a post-doctoral fellow with the Hong Kong University of Science and Technology, Hong Kong. From 2020 to 2022, she was a researcher with Huawei Hong Kong Research Center, Hong Kong. Her research interests include data privacy and security, distributed machine



learning, CCS, and INFOCOM. His research interests include computer science in the areas of network security, cloud security. He was a reviewer for many journals and conferences, such as *IEEE Transactions on Information Forensics*, *IEEE Transactions on Computers*, *IEEE/ACM Transactions on Networking*.

Jing Chen received the PhD degree in computer science from the Huazhong University of Science and Technology, Wuhan. He was a full professor with Wuhan University from 2015. He has authored or coauthored more than 100 research papers in many international journals and conferences, such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, *USENIX Security*, *CCS*, and *INFOCOM*. His research interests include computer science in the areas of network security, cloud security. He was a reviewer for many journals and conferences, such as *IEEE Transactions on Information Forensics*, *IEEE Transactions on Computers*, *IEEE/ACM Transactions on Networking*.



Juntong Gao (Graduate Student Member, IEEE) received the BE degree in computer science and technology from the Chongqing University of Posts and Telecommunications, Chongqing, China, in 2020, and the MS degree from Wuhan University, Wuhan, China, in 2023. She is currently working toward the PhD degree with the School of Cyber Science and Engineering, Wuhan University. Her research interests focus on privacy-preserving machine learning and inference.



network, cloud computing and mobile computing.

Ruiying Du received the BS, MS, and PhD degrees in computer science from Wuhan University, Wuhan, China, in 1987, 1994 and 2008, respectively. She is currently a professor with the School of Cyber Science and Engineering, Wuhan University. She has authored or coauthored more than 80 research papers in many international journals and conferences, such as *IEEE Transactions on Parallel and Distributed Systems*, *International Journal of Parallel and Distributed System*, *INFOCOM*, *SECON*, *TrustCom*, *NSS*. Her research interests include network security, wireless